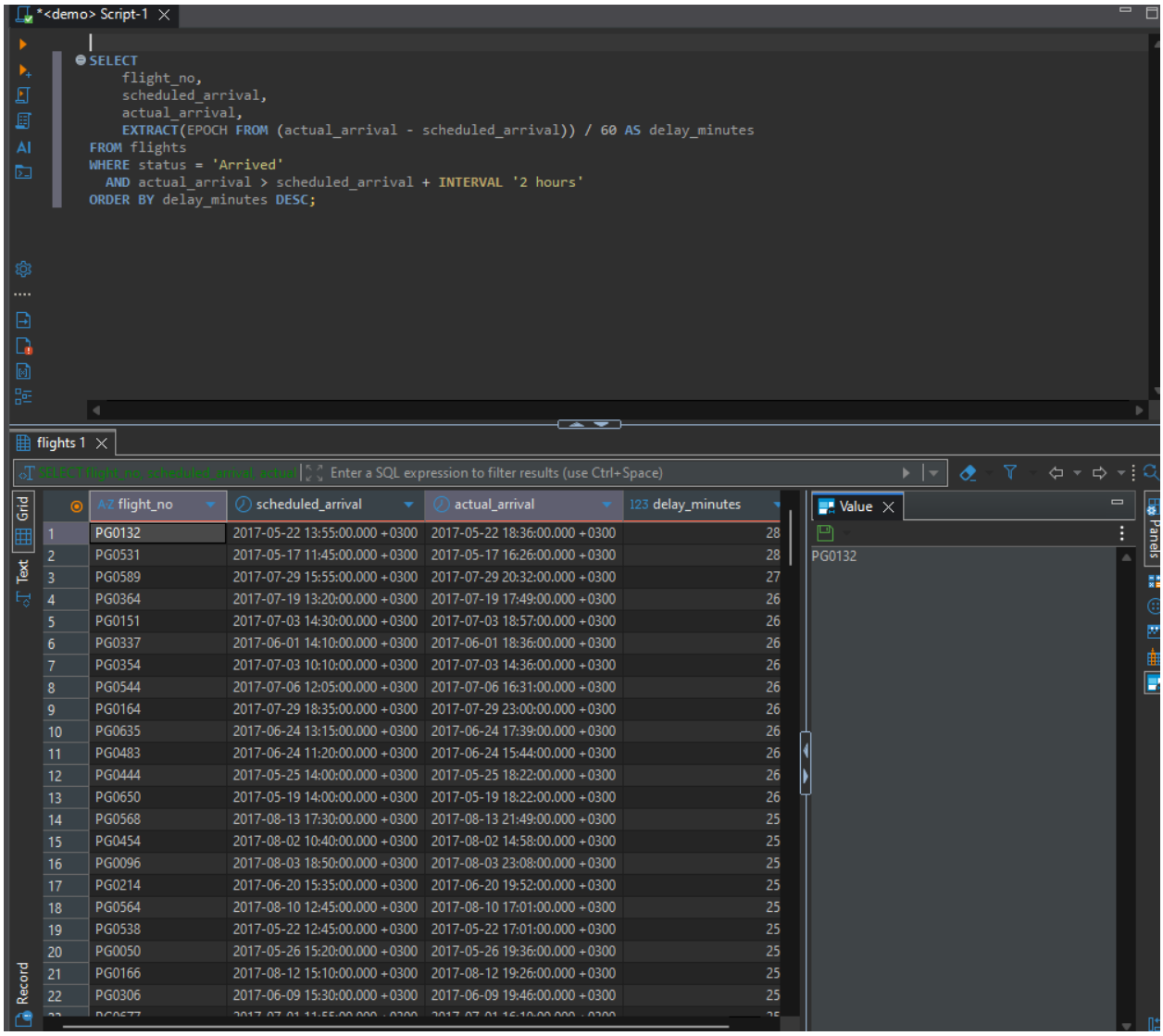
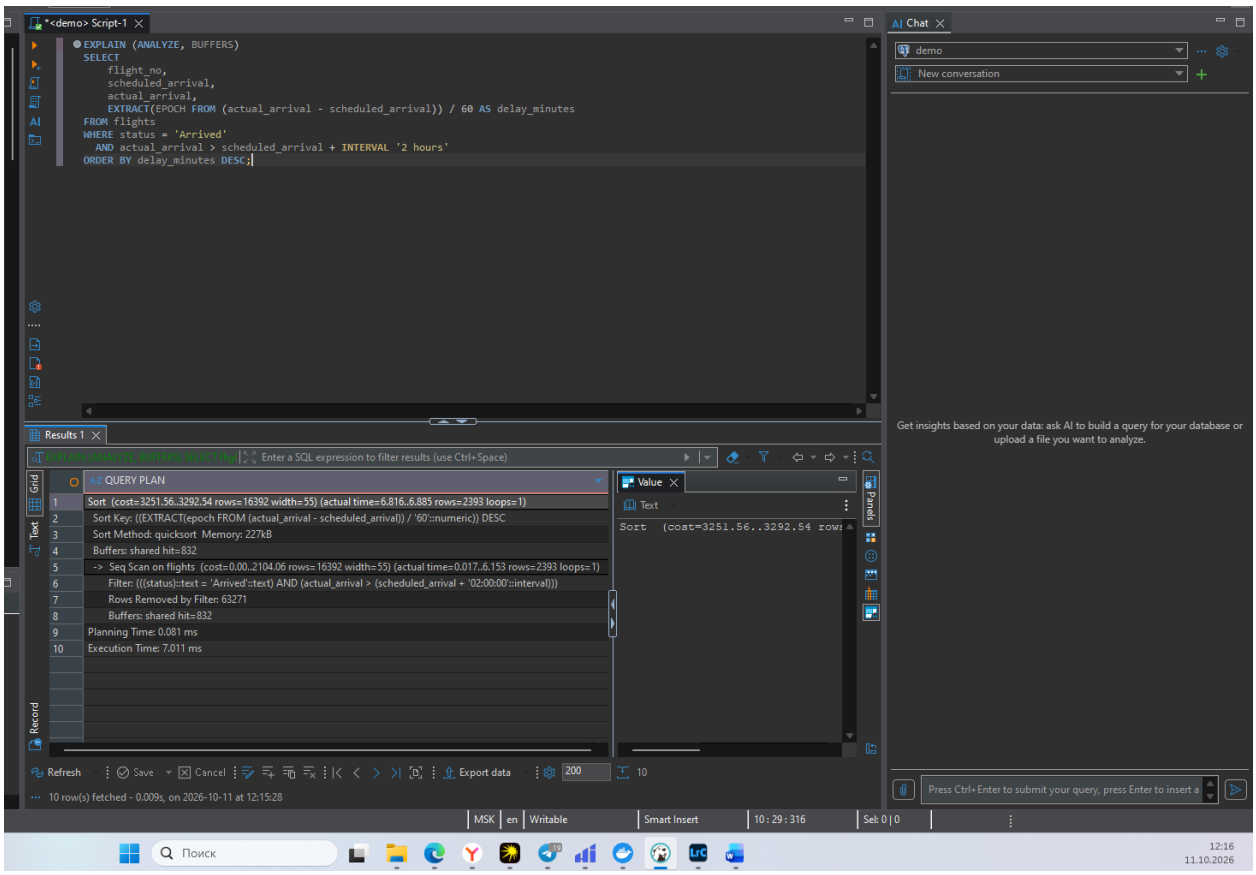
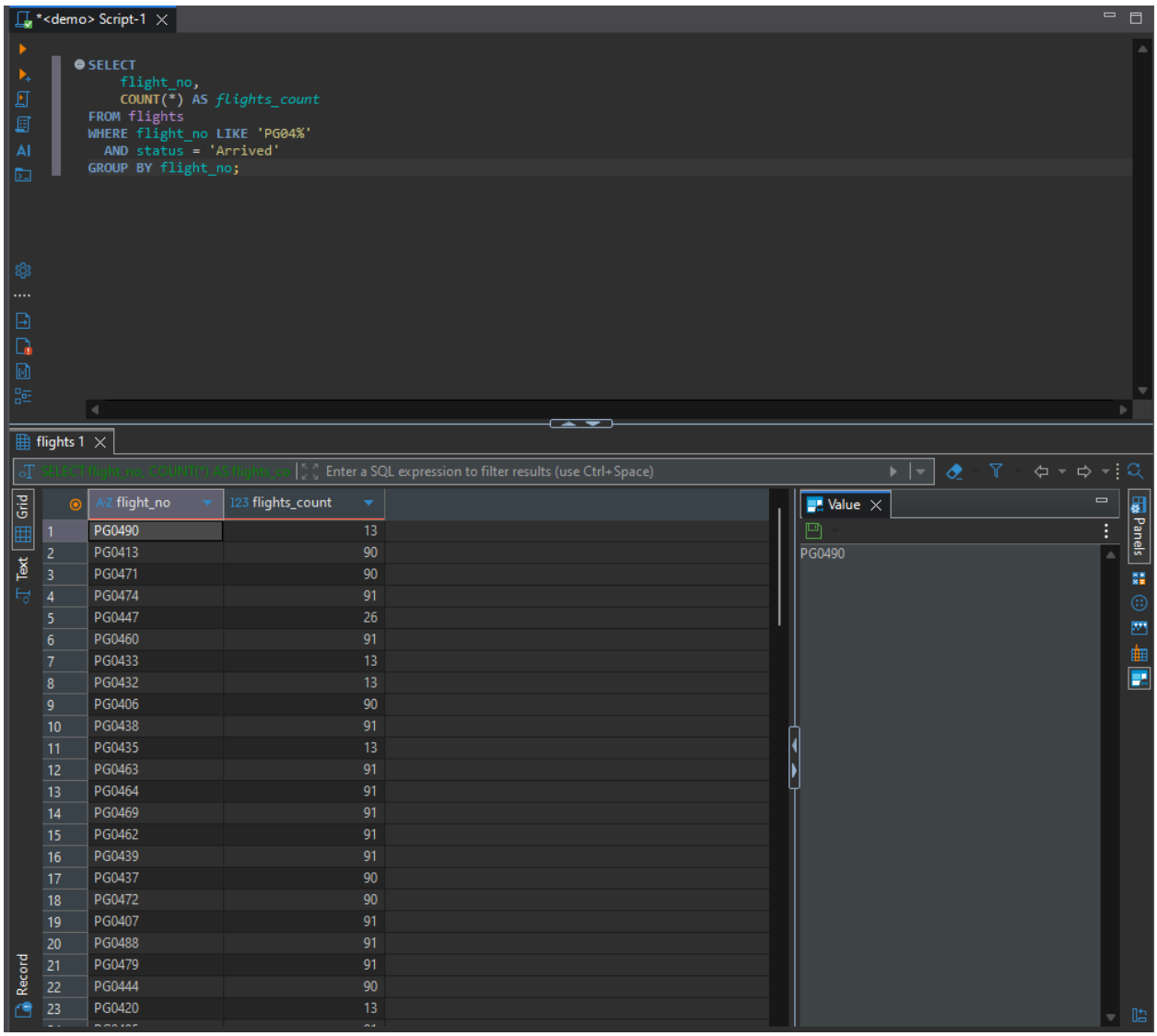
Задача 1.2

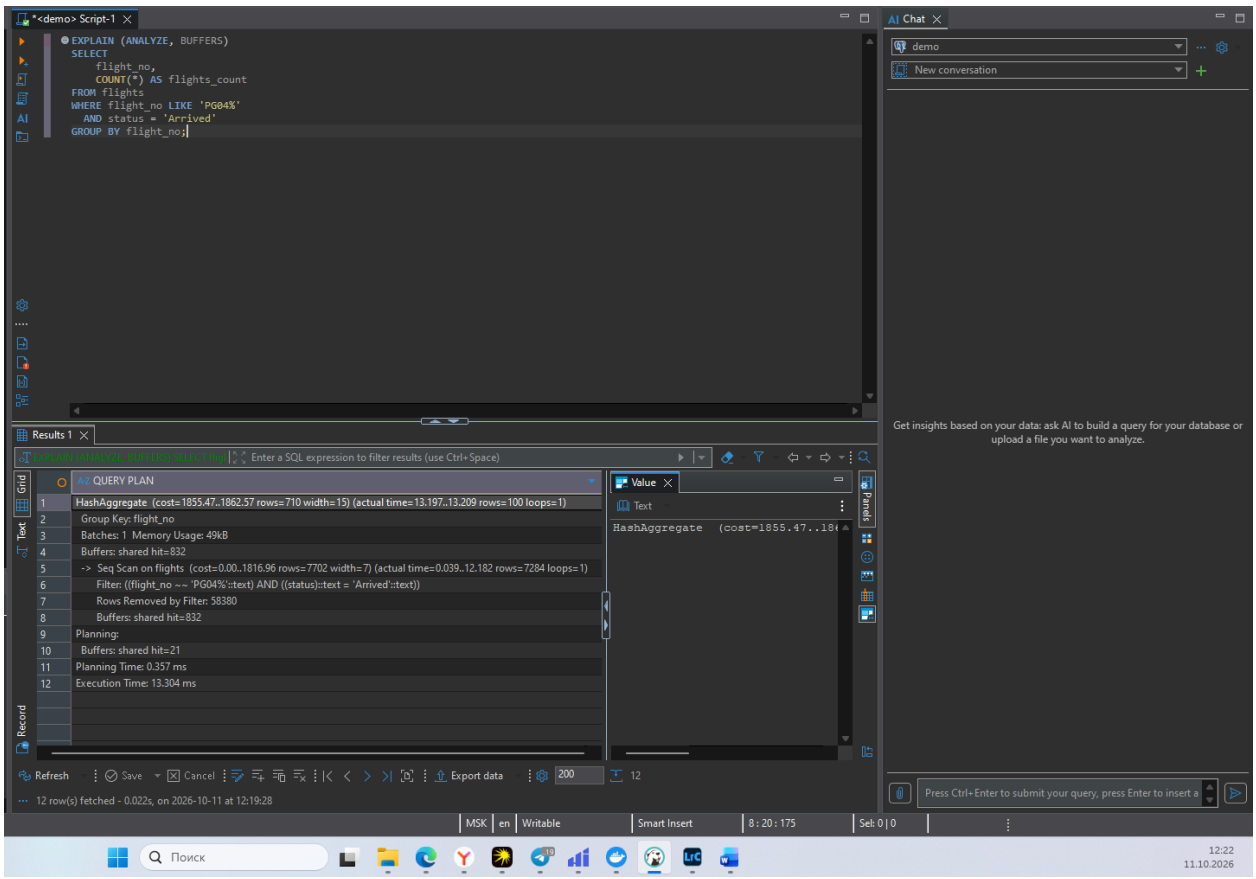

*<demo> Script-1
SELECT
flight_no,
scheduled_arrival,
actual_arrival,
EXTRACT(EPOCH FROM (actual_arrival - scheduled_arrival)) / 60 AS delay_minutes
FROM flights
WHERE status = 'Arrived'
AND actual_arrival > scheduled_arrival + INTERVAL '2 hours'
ORDER BY delay_minutes DESC;
flights 1
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
Value
PG0132
flight_no
scheduled_arrival
actual_arrival
delay_minutes
1 PG0132 2017-05-22 13:55:00.000 +0300 2017-05-22 18:36:00.000 +0300 28
2 PG0531 2017-05-17 11:45:00.000 +0300 2017-05-17 16:26:00.000 +0300 28
3 PG0589 2017-07-29 15:55:00.000 +0300 2017-07-29 20:32:00.000 +0300 27
4 PG0364 2017-07-19 13:20:00.000 +0300 2017-07-19 17:49:00.000 +0300 26
5 PG0151 2017-07-03 14:30:00.000 +0300 2017-07-03 18:57:00.000 +0300 26
6 PG0337 2017-06-01 14:10:00.000 +0300 2017-06-01 18:36:00.000 +0300 26
7 PG0354 2017-07-03 10:10:00.000 +0300 2017-07-03 14:36:00.000 +0300 26
8 PG0544 2017-07-06 12:05:00.000 +0300 2017-07-06 16:31:00.000 +0300 26
9 PG0164 2017-07-29 18:35:00.000 +0300 2017-07-29 23:00:00.000 +0300 26
10 PG0635 2017-06-24 13:15:00.000 +0300 2017-06-24 17:39:00.000 +0300 26
11 PG0483 2017-06-24 11:20:00.000 +0300 2017-06-24 15:44:00.000 +0300 26
12 PG0444 2017-05-25 14:00:00.000 +0300 2017-05-25 18:22:00.000 +0300 26
13 PG0650 2017-05-19 14:00:00.000 +0300 2017-05-19 18:22:00.000 +0300 26
14 PG0568 2017-08-13 17:30:00.000 +0300 2017-08-13 21:49:00.000 +0300 25
15 PG0454 2017-08-02 10:40:00.000 +0300 2017-08-02 14:58:00.000 +0300 25
16 PG0096 2017-08-03 18:50:00.000 +0300 2017-08-03 23:08:00.000 +0300 25
17 PG0214 2017-06-20 15:35:00.000 +0300 2017-06-20 19:52:00.000 +0300 25
18 PG0564 2017-08-10 12:45:00.000 +0300 2017-08-10 17:01:00.000 +0300 25
19 PG0538 2017-05-22 12:45:00.000 +0300 2017-05-22 17:01:00.000 +0300 25
20 PG0050 2017-05-26 15:20:00.000 +0300 2017-05-26 19:36:00.000 +0300 25
21 PG0166 2017-08-12 15:10:00.000 +0300 2017-08-12 19:26:00.000 +0300 25
22 PG0306 2017-06-09 15:30:00.000 +0300 2017-06-09 19:46:00.000 +0300 25



Аналитическое резюме: Запрос не использовал индексы, выполнив полное последовательное сканирование таблицы (Seq Scan on flights), так как фильтрация происходит по вычисляемой разнице между временем прибытия по расписанию и фактическим. Физическое чтение с диска не производилось, поскольку все необходимые 832 блока данных были успешно найдены в оперативной памяти (кэше), на что указывает Buffers: shared hit=832. Для итоговой сортировки результатов по убыванию времени задержки применялся алгоритм quicksort, который полностью отработал в памяти, потребовав 227kB

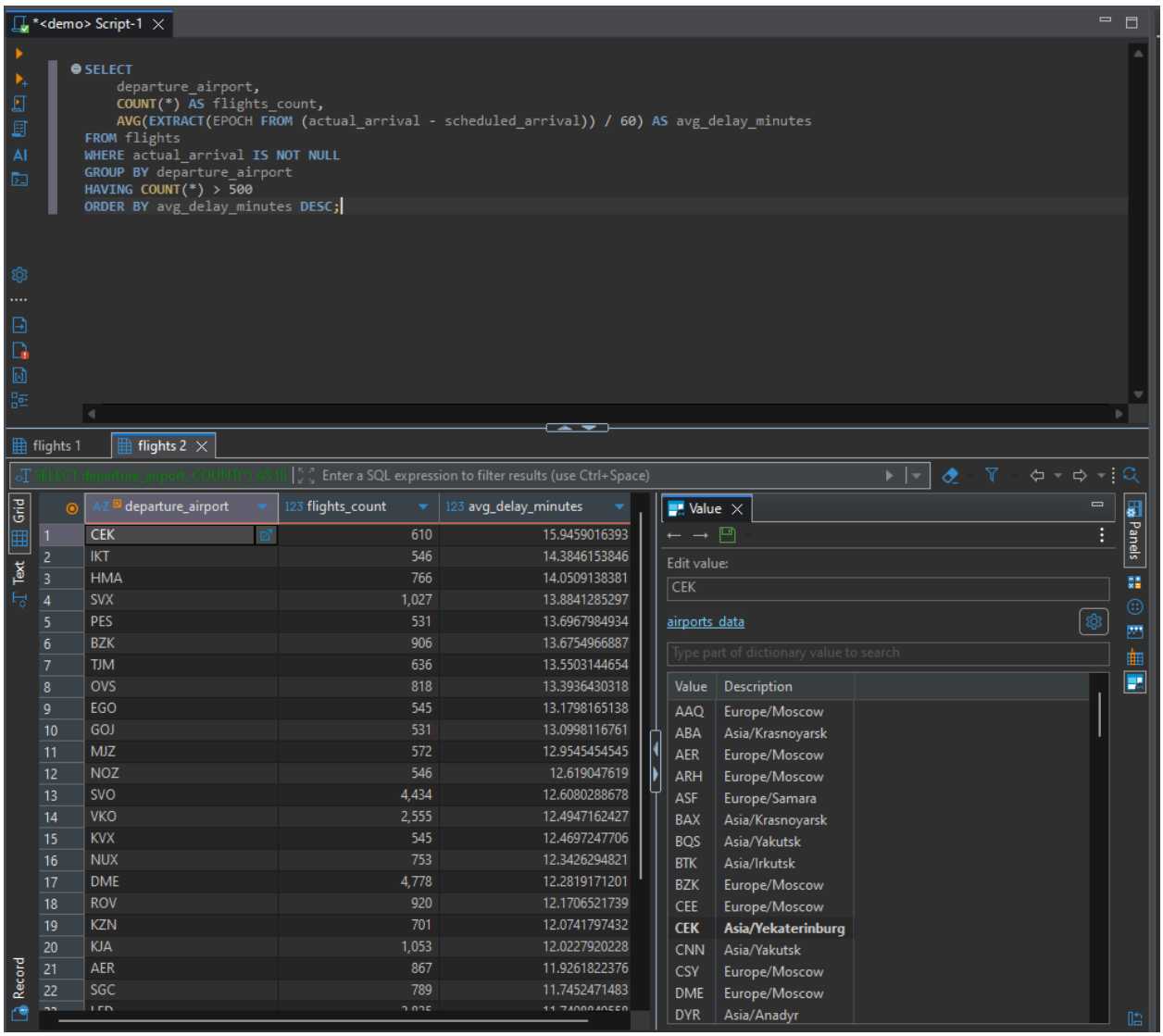
Задача 1.3

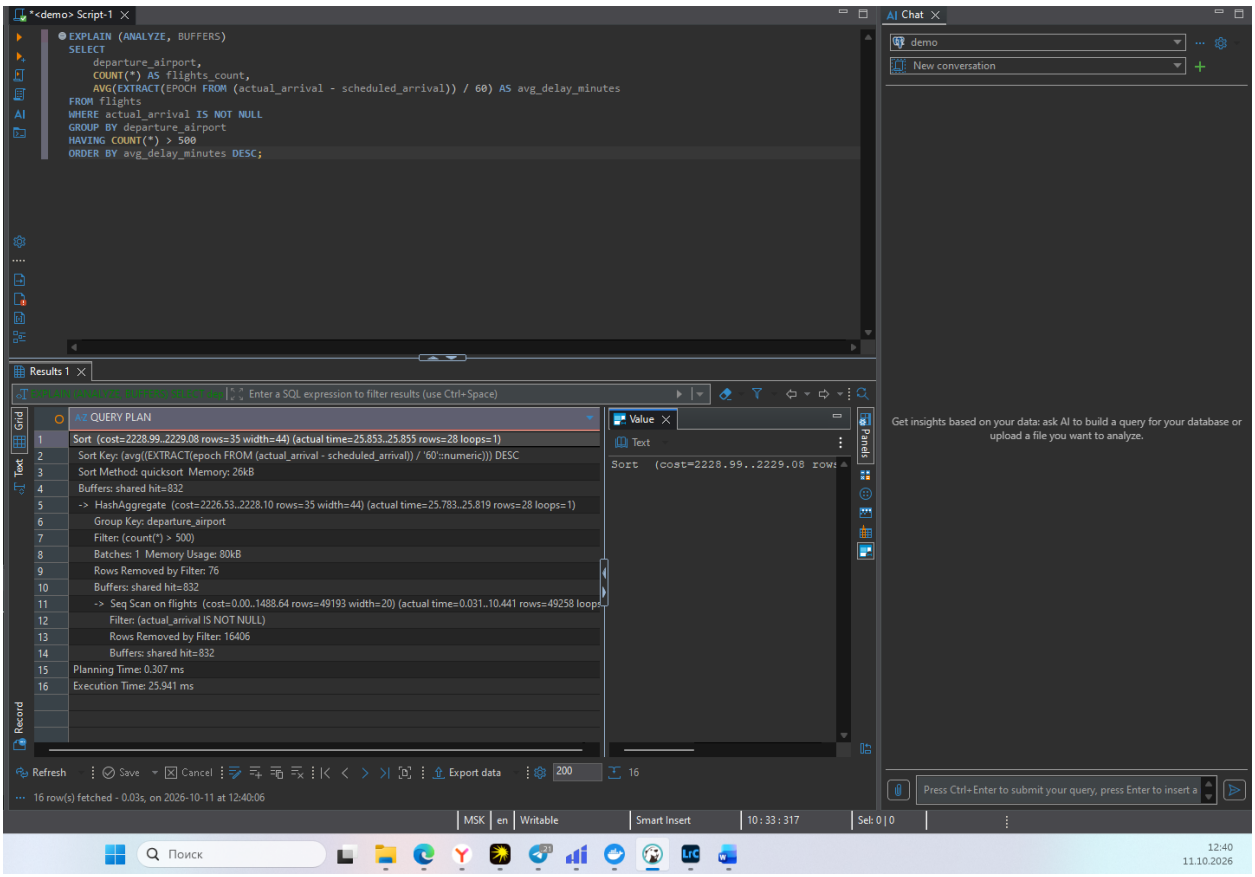

*<demo> Script-1
SELECT
flight_no,
COUNT(*) AS flights_count
FROM flights
WHERE flight_no LIKE 'PG04%'
AND status = 'Arrived'
GROUP BY flight_no;
flights 1
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
Panels
Value
PG0490
flight_no
flights_count
1 PG0490 13
2 PG0413 90
3 PG0471 90
4 PG0474 91
5 PG0447 26
6 PG0460 91
7 PG0433 13
8 PG0432 13
9 PG0406 90
10 PG0438 91
11 PG0435 13
12 PG0463 91
13 PG0464 91
14 PG0469 91
15 PG0462 91
16 PG0439 91
17 PG0437 90
18 PG0472 90
19 PG0407 91
20 PG0488 91
21 PG0479 91
22 PG0444 90
23 PG0420 13



Аналитическое резюме: при выполнении запроса индексы не применялись, поэтому было выполнено полное последовательное сканирование таблицы (Seq Scan on flights). Физическое чтение с диска не производилось, поскольку все нужные 832 блока данных уже находились и были прочитаны из кэша (shared hit=832). Группировка и подсчет рейсов осуществлялись с помощью узла HashAggregate, который выполнил операцию в оперативной памяти, потратив на это 49kB

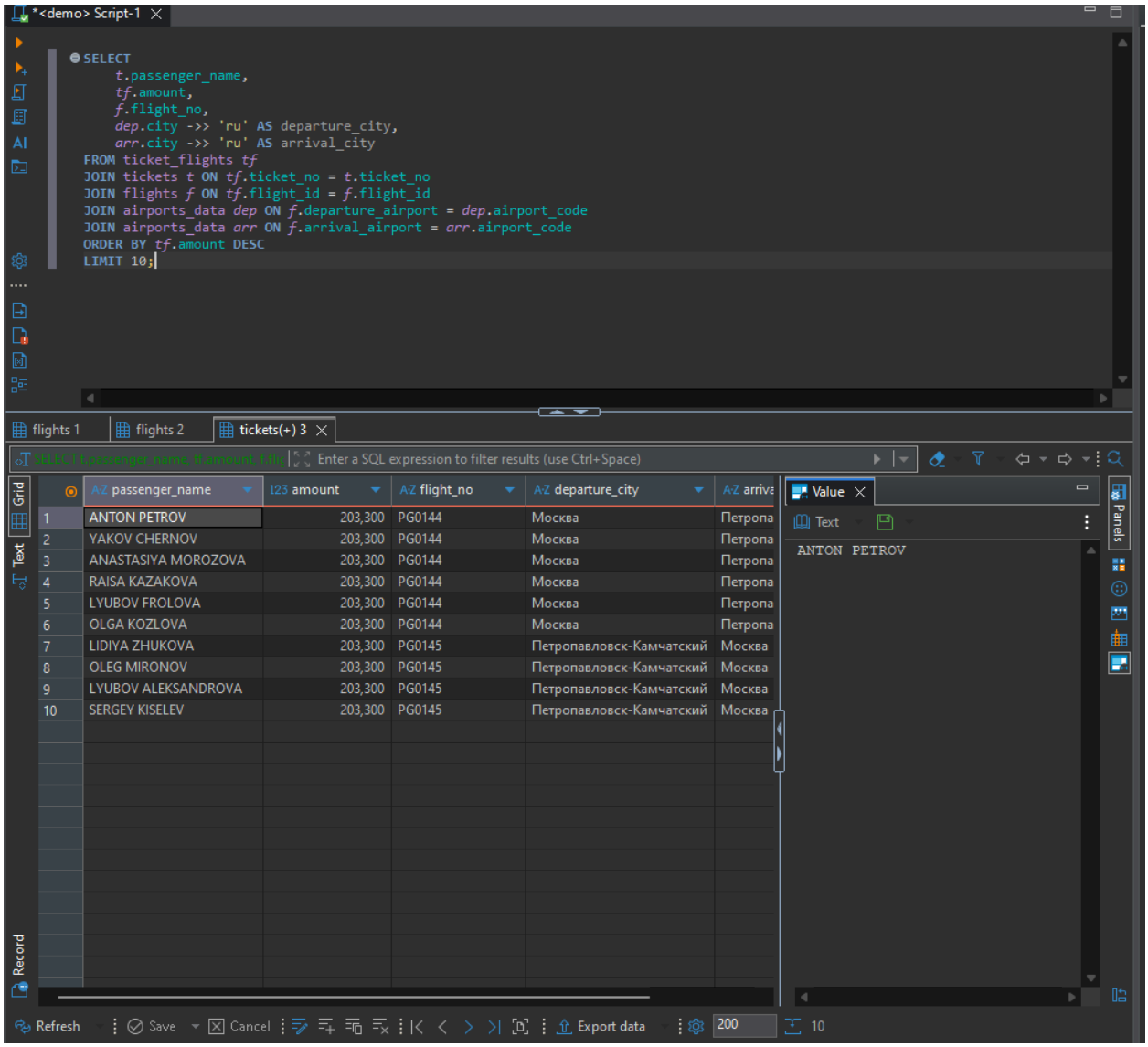
Задача 2.1

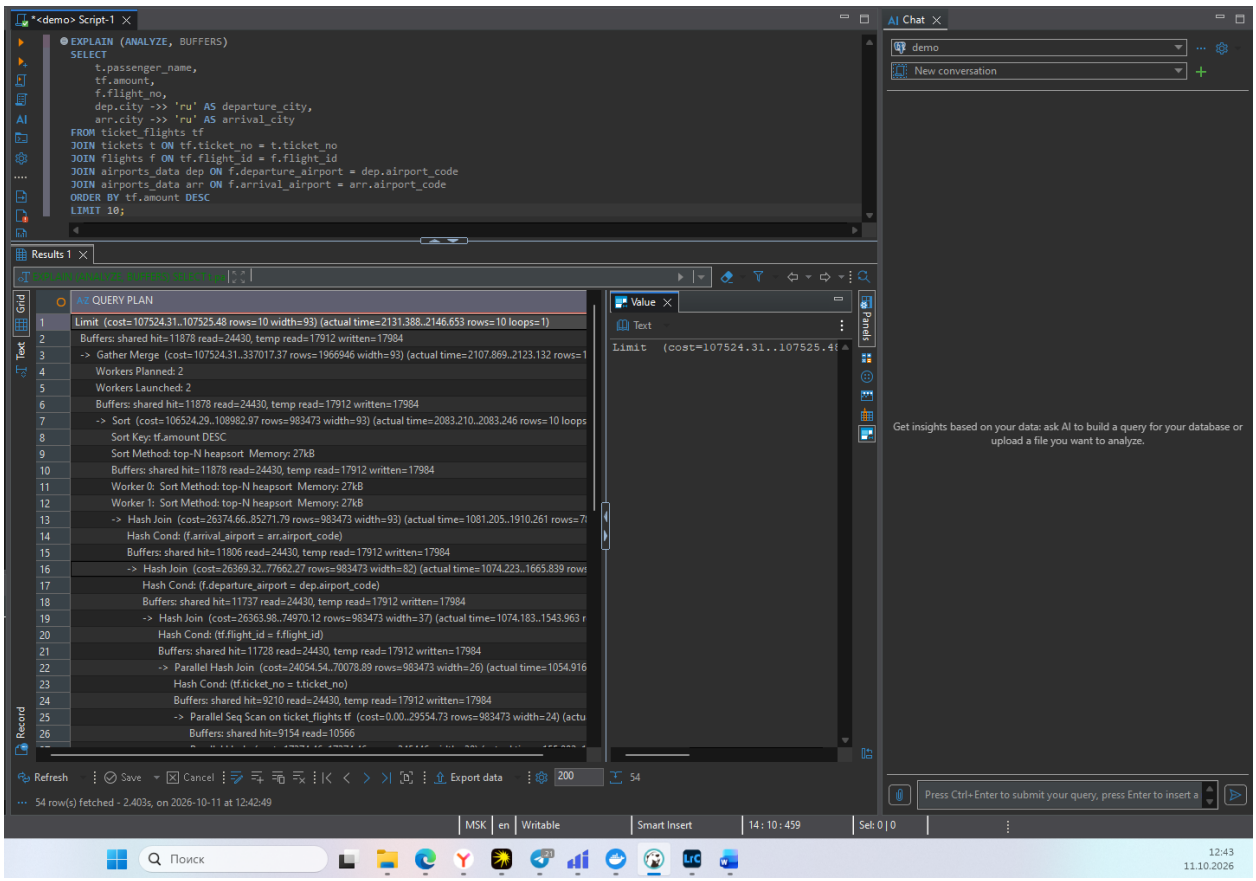

*<demo> Script-1
SELECT
departure_airport,
COUNT(*) AS flights_count,
AVG(EXTRACT(EPOCH FROM (actual_arrival - scheduled_arrival)) / 60) AS avg_delay_minutes
FROM flights
WHERE actual_arrival IS NOT NULL
GROUP BY departure_airport
HAVING COUNT(*) > 500
ORDER BY avg_delay_minutes DESC;
flights 1
flights 2
Enter a SQL expression to filter results (use Ctrl+Space)
departure_airport
flights_count
avg_delay_minutes
1 CEK 610 15.9459016393
2 IKT 546 14.3846153846
3 HMA 766 14.0509138381
4 SVX 1,027 13.8841285297
5 PES 531 13.6967984934
6 BZK 906 13.6754966887
7 TJM 636 13.5503144654
8 OVS 818 13.3936430318
9 EGO 545 13.1798165138
10 GOJ 531 13.0998116761
11 MJZ 572 12.9545454545
12 NOZ 546 12.619047619
13 SVO 4,434 12.6080288678
14 VKO 2,555 12.4947162427
15 KVX 545 12.4697247706
16 NUX 753 12.3426294821
17 DME 4,778 12.2819171201
18 ROV 920 12.1706521739
19 KZN 701 12.0741797432
20 KJA 1,053 12.0227920228
21 AER 867 11.9261822376
22 SGC 789 11.7452471483
Value
Edit value:
CEK
airports_data
Type part of dictionary value to search
Value Description
AAQ Europe/Moscow
ABA Asia/Krasnoyarsk
AER Europe/Moscow
ARH Europe/Moscow
ASF Europe/Samara
BAX Asia/Krasnoyarsk
BQS Asia/Yakutsk
BTK Asia/Irkutsk
BZK Europe/Moscow
CEE Europe/Moscow
CEK Asia/Yekaterinburg
CNN Asia/Yakutsk
CSY Europe/Moscow
DME Europe/Moscow
DYR Asia/Anadyr
Grid
Text
Record
Panels

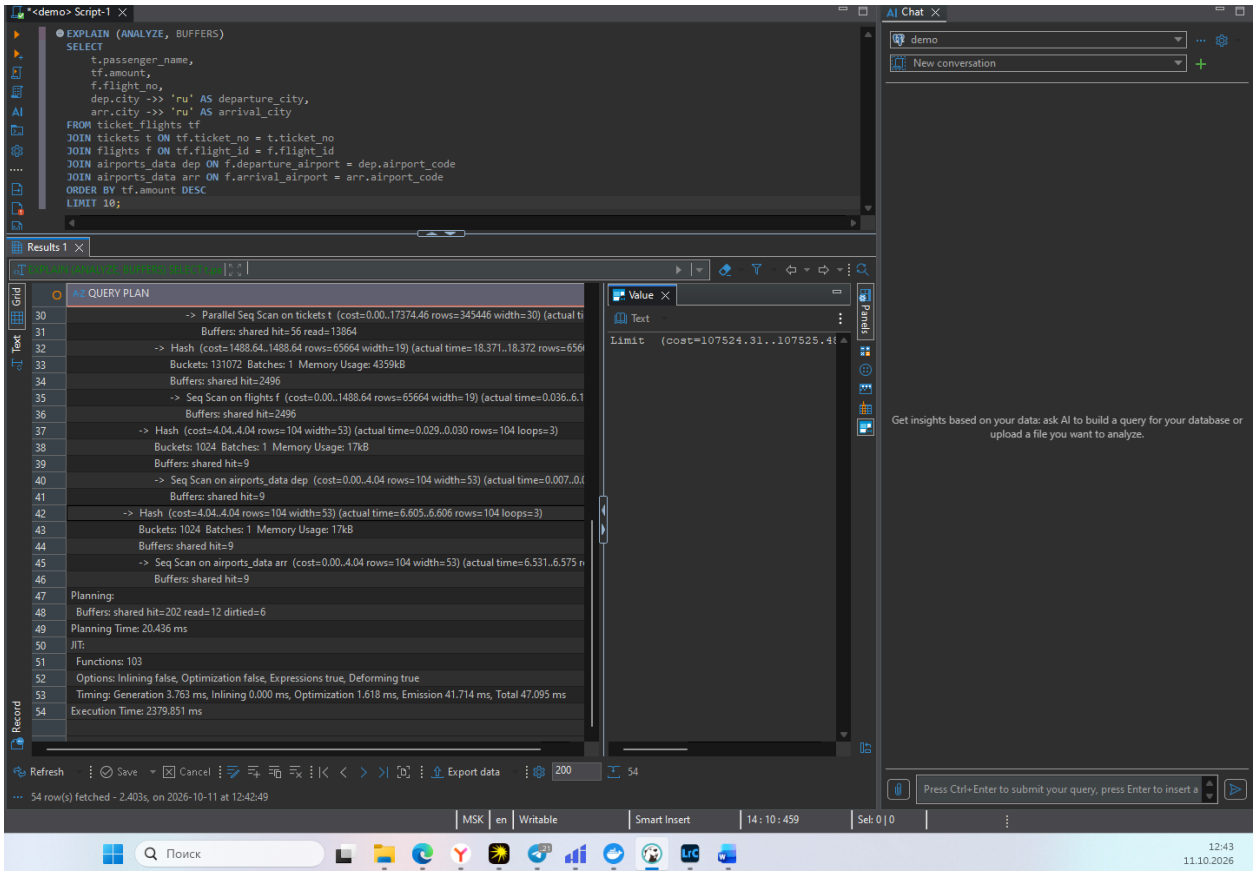


Аналитическое резюме: при выполнении запроса индекс не использовался, база данных осуществила полное последовательное сканирование таблицы (Seq Scan on flights). Физического обращения к диску не потребовалось, так как все 832 необходимых блока данных были найдены и прочитаны из кэша (shared hit=832). Последующие операции группировки (HashAggregate) и сортировки результатов (quicksort) были выполнены в оперативной памяти, заняв 80kB и 26kB соответственно

Задача 3.1


*<demo> Script-1
SELECT
t.passenger_name,
tf.amount,
f.flight_no,
dep.city ->> 'ru' AS departure_city,
arr.city ->> 'ru' AS arrival_city
FROM ticket_flights tf
JOIN tickets t ON tf.ticket_no = t.ticket_no
JOIN flights f ON tf.flight_id = f.flight_id
JOIN airports_data dep ON f.departure_airport = dep.airport_code
JOIN airports_data arr ON f.arrival_airport = arr.airport_code
ORDER BY tf.amount DESC
LIMIT 10;
flights 1
flights 2
tickets(+) 3
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
passenger_name
amount
flight_no
departure_city
arriva
1 ANTON PETROV 203,300 PG0144 Москва Петропа
2 YAKOV CHERNOV 203,300 PG0144 Москва Петропа
3 ANASTASIYA MOROZOVA 203,300 PG0144 Москва Петропа
4 RAISA KAZAKOVA 203,300 PG0144 Москва Петропа
5 LYUBOV FROLOVA 203,300 PG0144 Москва Петропа
6 OLGA KOZLOVA 203,300 PG0144 Москва Петропа
7 LIDIYA ZHUKOVA 203,300 PG0145 Петропавловск-Камчатский Москва
8 OLEG MIRONOV 203,300 PG0145 Петропавловск-Камчатский Москва
9 LYUBOV ALEKSANDROVA 203,300 PG0145 Петропавловск-Камчатский Москва
10 SERGEY KISELEV 203,300 PG0145 Петропавловск-Камчатский Москва
Value
Text
ANTON PETROV
Panels
Refresh
Save
Cancel
Export data
200
10

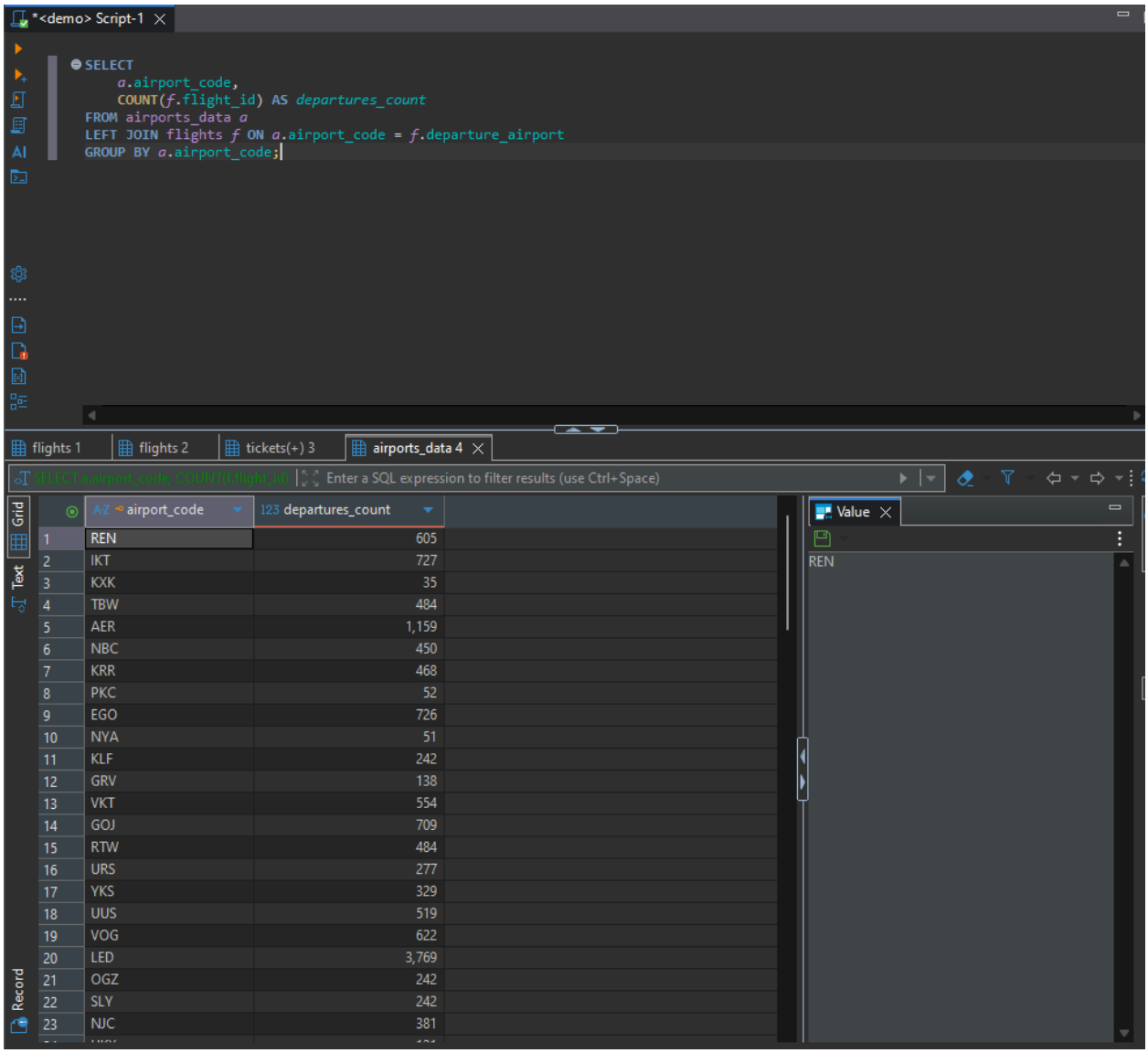


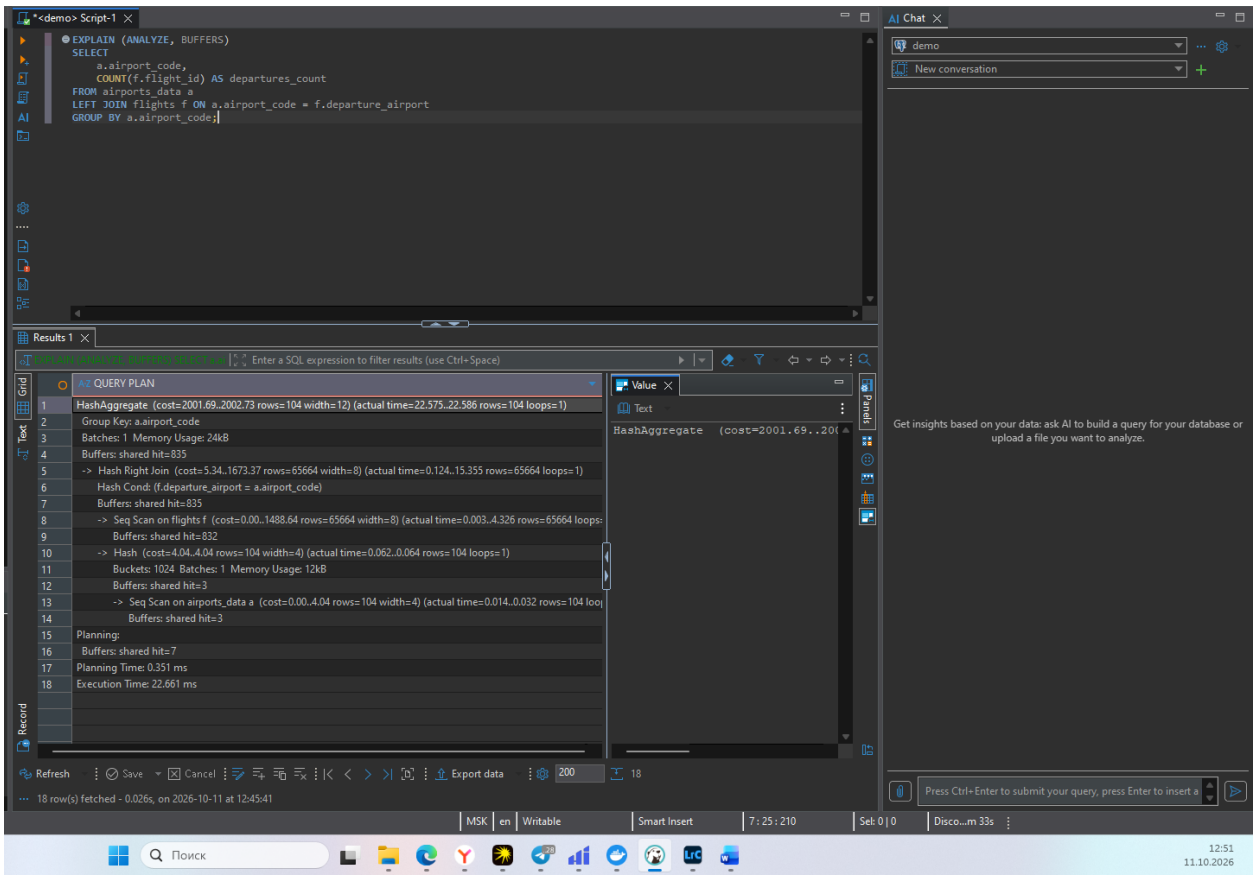


Аналитическое резюме: Запрос выполнялся в несколько потоков с использованием полного последовательного сканирования всех участвующих таблиц (Parallel Seq Scan и Seq Scan), без применения индексов. База данных активно работала как с физическим

диском, прочитав более 24 тысяч блоков (read=24430), так и с кэшем, найдя почти 12 тысяч блоков в оперативной памяти (shared hit=11878). Кроме того, из-за большого объема объединяемых данных при выполнении Hash Join база данных была вынуждена сбрасывать часть вычислений во временные файлы на диске (temp written=17984), тогда как финальная сортировка 10 строк успешно завершилась в оперативной памяти с помощью алгоритма top-N heapsort

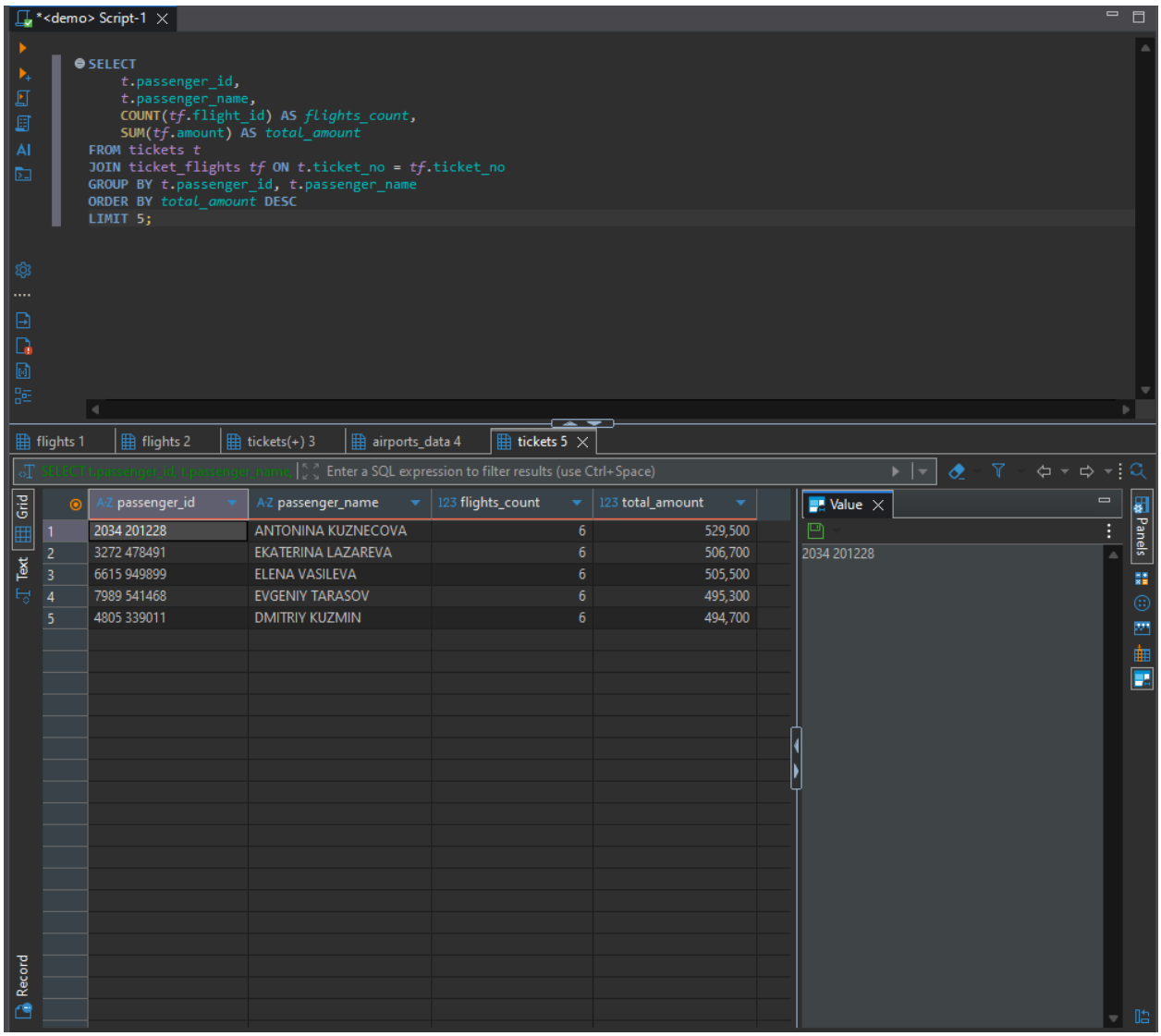
Задача 3.2

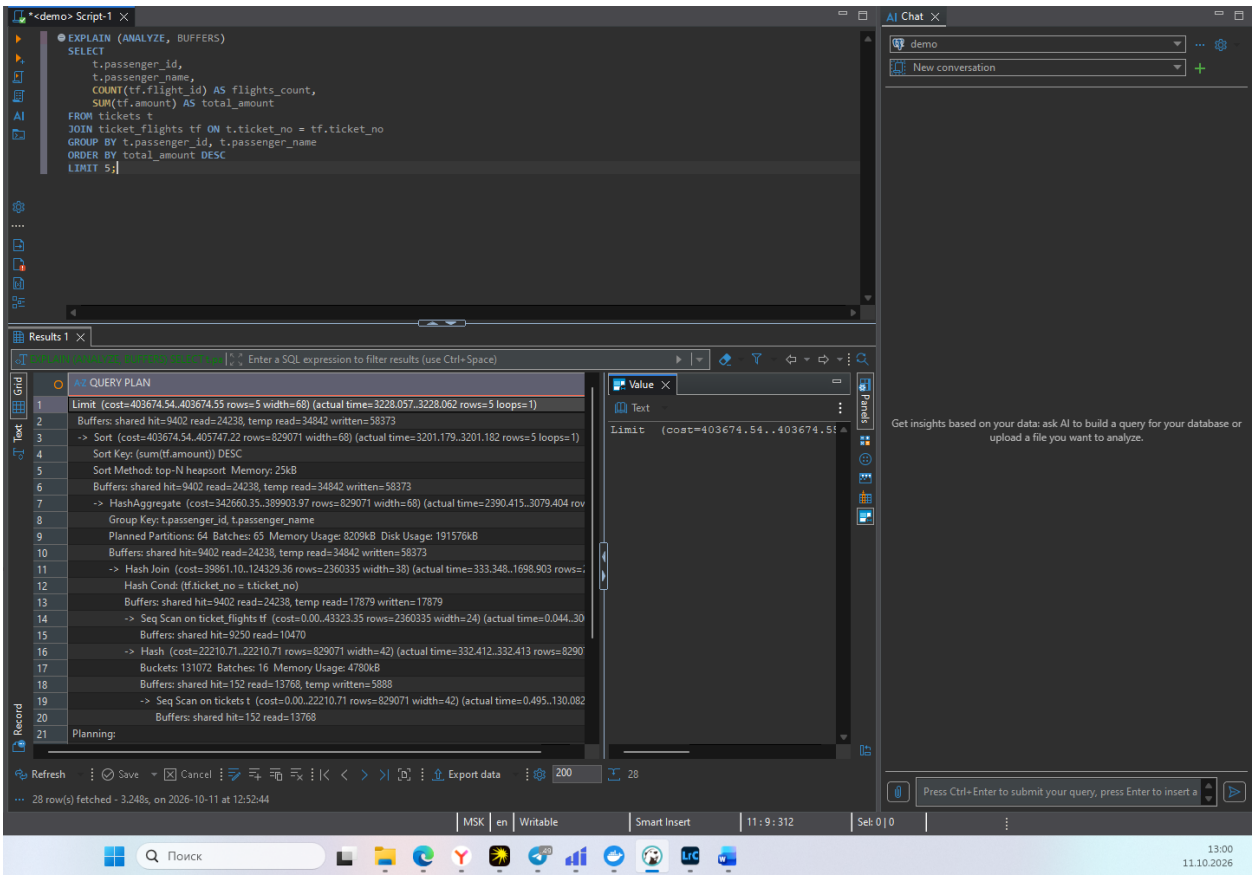


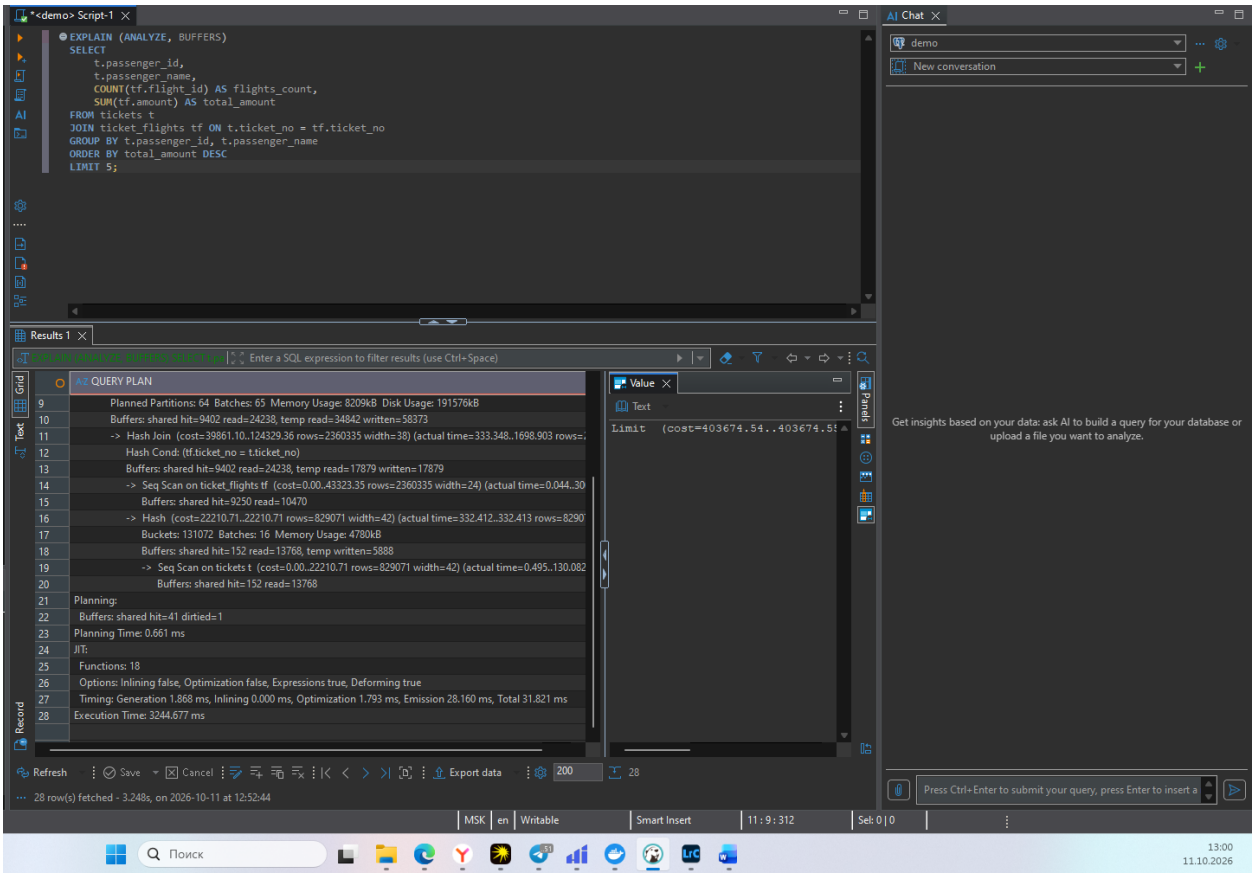


Аналитическое резюме: при выполнении запроса индексы не применялись, планировщик предпочел выполнить полное последовательное сканирование (Seq Scan) обеих таблиц — flights и airports_data. Обращений к физическому диску не было, так как все 835 необходимых блоков данных были успешно найдены и прочитаны из кэша (shared hit=835). Само соединение таблиц (Hash Right Join) и последующая агрегация данных (HashAggregate) были выполнены в оперативной памяти

Задача 3.3


*<demo> Script-1
SELECT
t.passenger_id,
t.passenger_name,
COUNT(tf.flight_id) AS flights_count,
SUM(tf.amount) AS total_amount
FROM tickets t
JOIN ticket_flights tf ON t.ticket_no = tf.ticket_no
GROUP BY t.passenger_id, t.passenger_name
ORDER BY total_amount DESC
LIMIT 5;
flights 1
flights 2
tickets(+) 3
airports_data 4
tickets 5
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
Panels
passenger_id
passenger_name
flights_count
total_amount
1 2034 201228 ANTONINA KUZNECOVA 6 529,500
2 3272 478491 EKATERINA LAZAREVA 6 506,700
3 6615 949899 ELENA VASILEVA 6 505,500
4 7989 541468 EVGENIY TARASOV 6 495,300
5 4805 339011 DMITRIY KUZMIN 6 494,700
Value
2034 201228

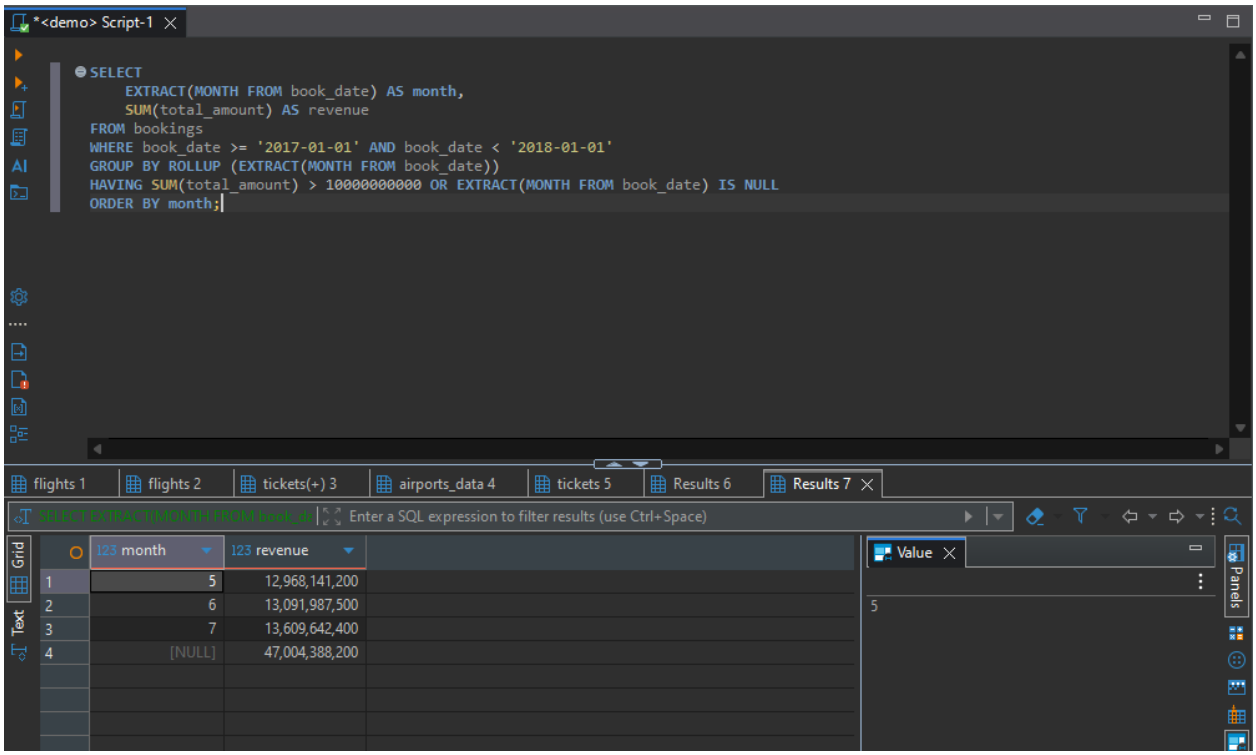


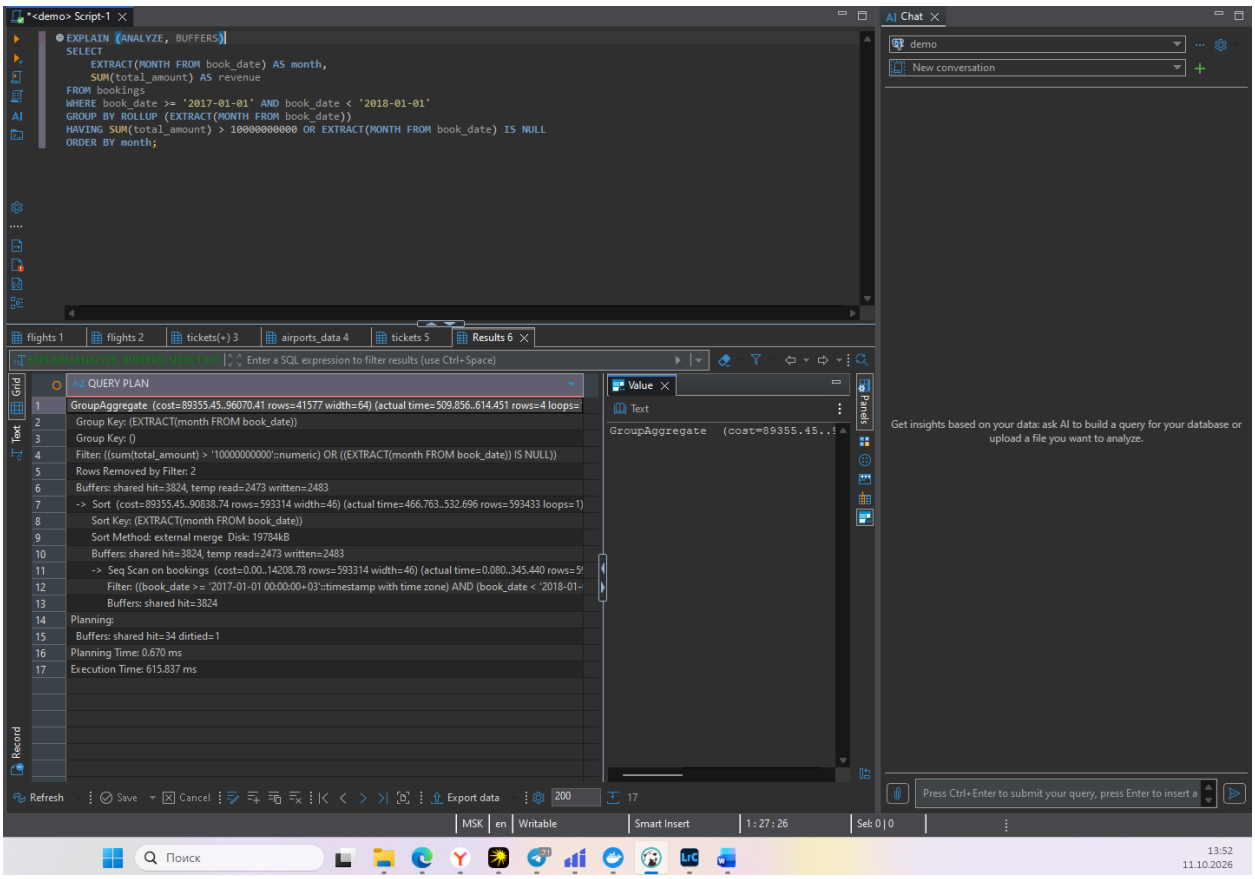


Аналитическое резюме: при выполнении запроса индексы не использовались, база данных была вынуждена применить полное последовательное сканирование (Seq Scan) обеих таблиц — tickets и ticket_flights. Запрос активно работал как с кэшем, прочитав

более 9 тысяч блоков из оперативной памяти (shared hit=9402), так и с физическим диском, считав более 24 тысяч блоков напрямую с накопителя (read=24238). Из-за большого объема обрабатываемых строк операции агрегации (HashAggregate) и соединения (Hash Join) не уместились в доступной оперативной памяти и потребовали интенсивной работы с временными файлами на диске (Disk Usage: 191576kB, высокие показатели temp read и temp written), тогда как итоговая сортировка 5 строк была выполнена в кэше алгоритмом top-N heapsort

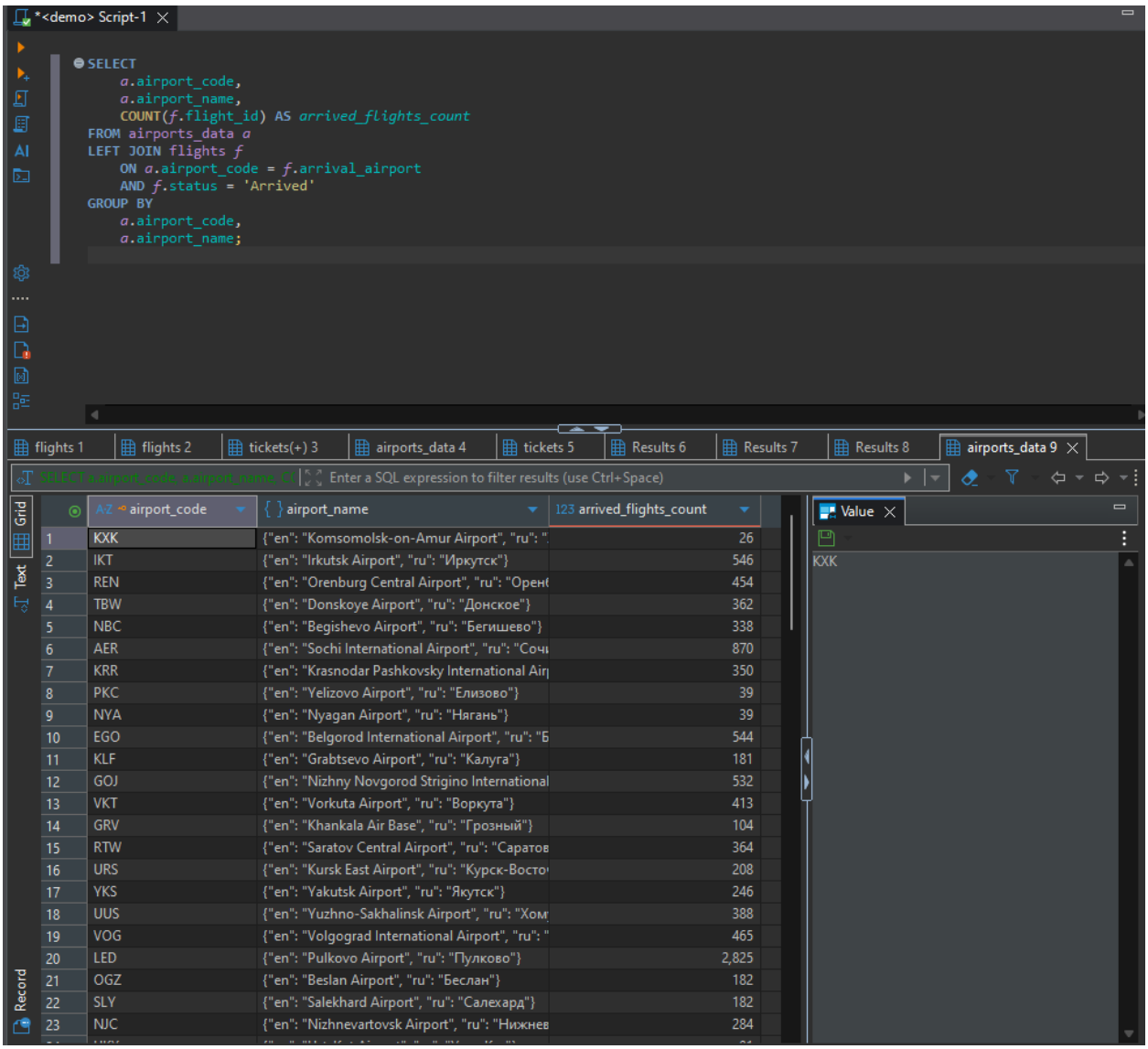
Задание 1

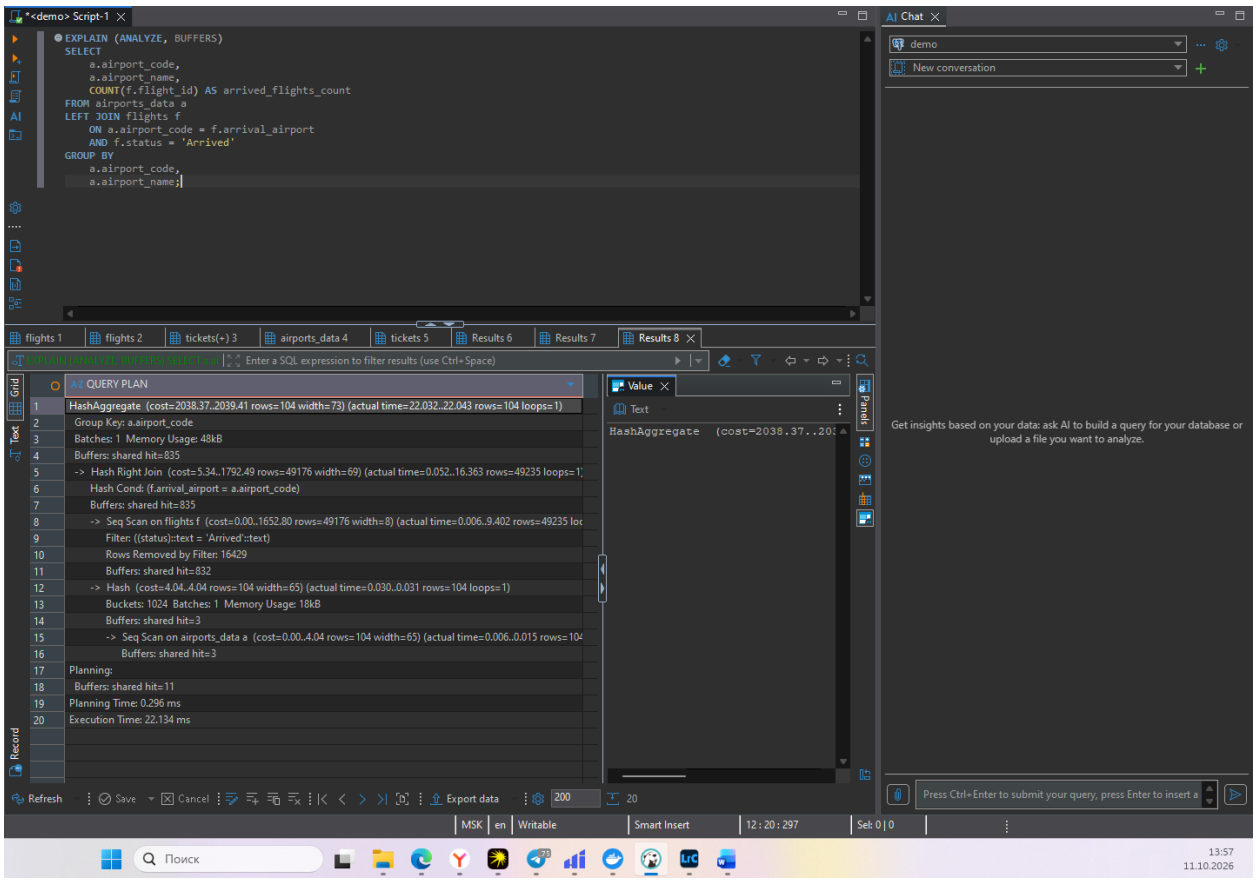




Аналитическое резюме: при выполнении запроса индексы не применялись, база данных осуществила полное последовательное сканирование таблицы bookings (Seq Scan). Первичное чтение данных происходило из кэша, так как необходимые 3824 блока были успешно найдены в оперативной памяти (shared hit=3824). Однако запрос активно работал с физическим диском на этапе подготовки к группировке: из-за нехватки оперативной памяти алгоритм сортировки был вынужден сбрасывать данные на диск (Sort Method: external merge Disk: 19784kB), что привело к интенсивному чтению и записи временных файлов (temp read=2473 written=2483)

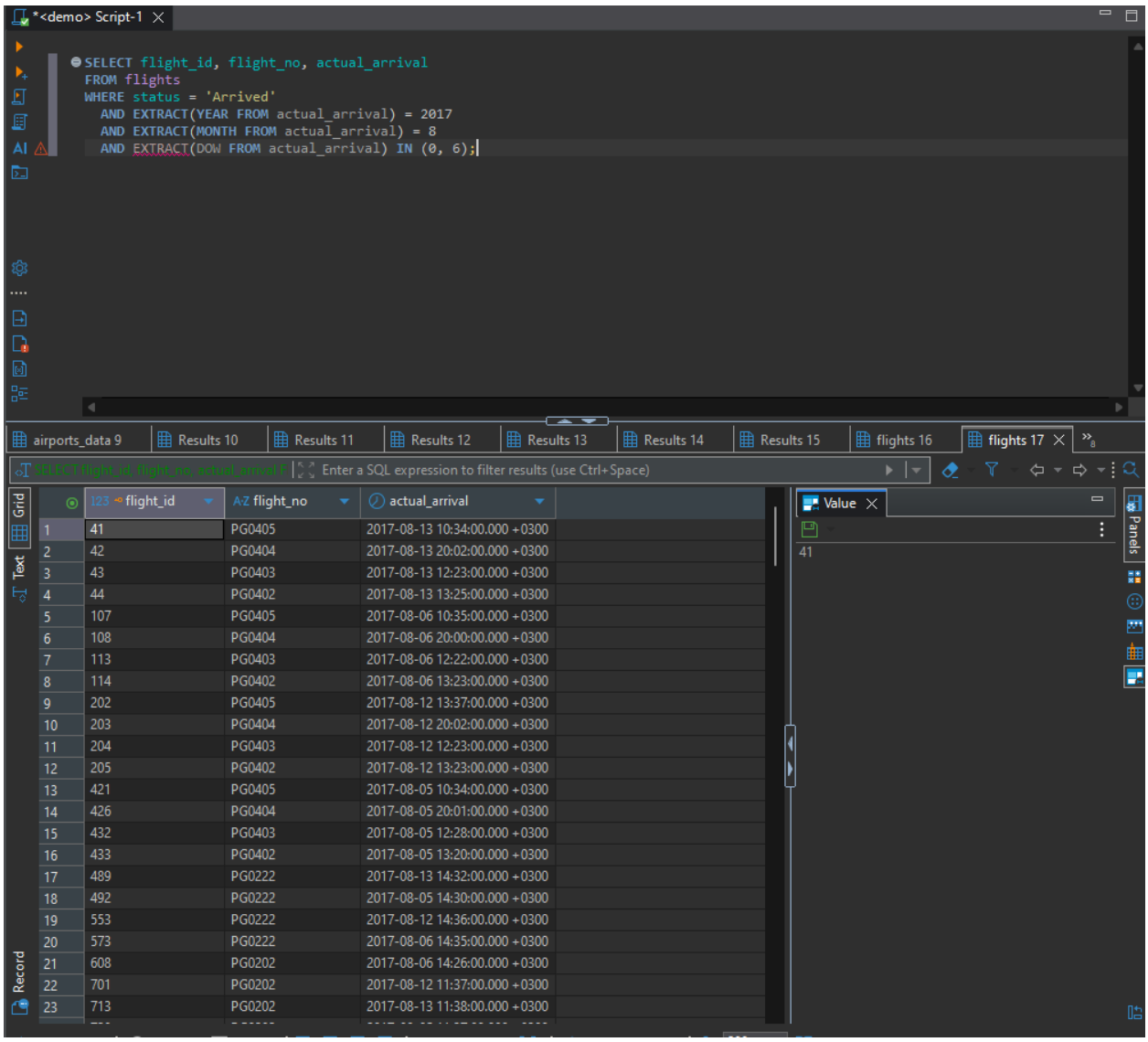
Задание 2

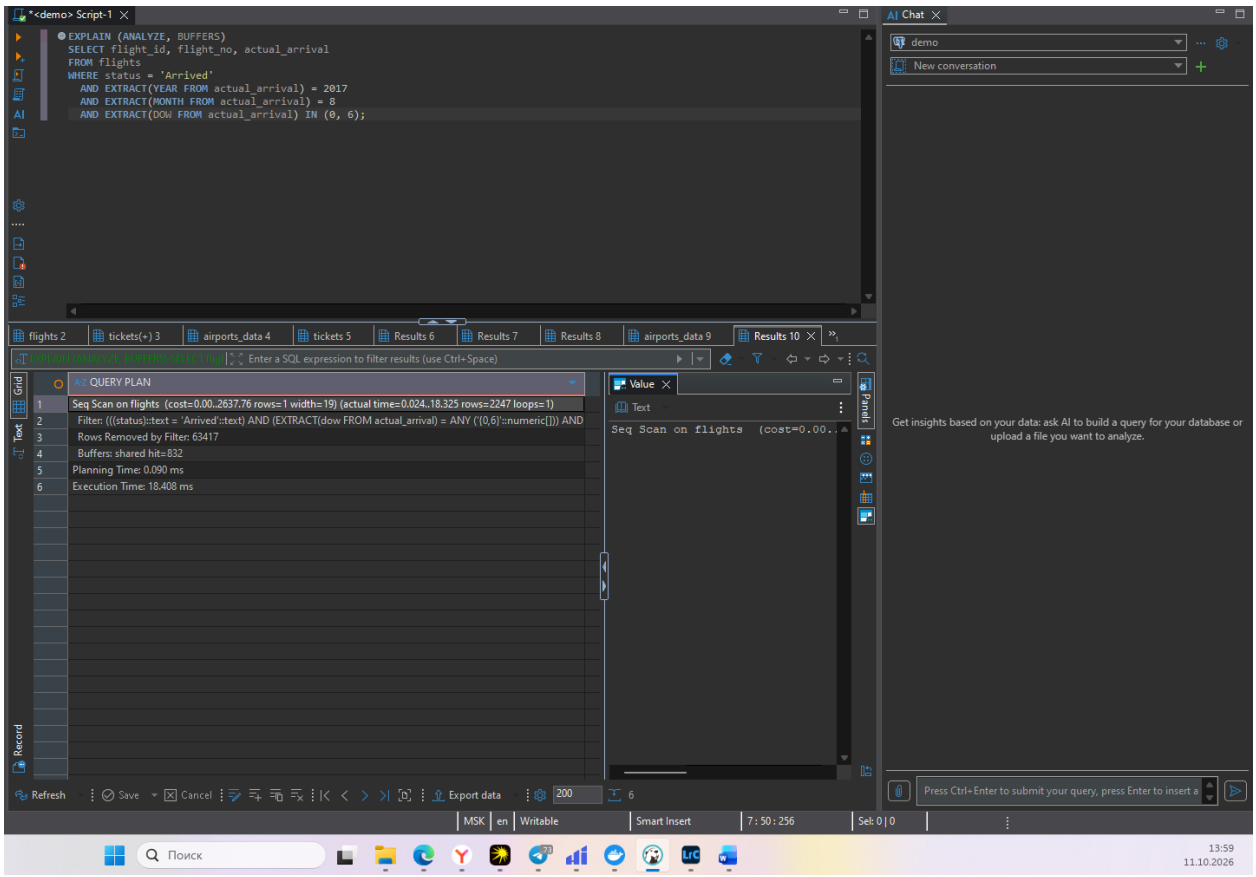

*<demo> Script-1
SELECT
a.airport_code,
a.airport_name,
COUNT(f.flight_id) AS arrived_flights_count
FROM airports_data a
LEFT JOIN flights f
ON a.airport_code = f.arrival_airport
AND f.status = 'Arrived'
GROUP BY
a.airport_code,
a.airport_name;
flights 1
flights 2
tickets(+) 3
airports_data 4
tickets 5
Results 6
Results 7
Results 8
airports_data 9
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
airport_code
airport_name
arrived_flights_count
1 KXK {"en": "Komsomolsk-on-Amur Airport", "ru": " 26
2 IKT {"en": "Irkutsk Airport", "ru": "Иркутск"} 546
3 REN {"en": "Orenburg Central Airport", "ru": "Орен 454
4 TBW {"en": "Donskoye Airport", "ru": "Донское"} 362
5 NBC {"en": "Begishevo Airport", "ru": "Бегишево"} 338
6 AER {"en": "Sochi International Airport", "ru": "Сочи 870
7 KRR {"en": "Krasnodar Pashkovsky International Air 350
8 PKC {"en": "Yelizovo Airport", "ru": "Елизово"} 39
9 NYA {"en": "Nyagan Airport", "ru": "Нягань"} 39
10 EGO {"en": "Belgorod International Airport", "ru": "Б 544
11 KLF {"en": "Grabtsevo Airport", "ru": "Калуга"} 181
12 GOJ {"en": "Nizhny Novgorod Strigino International 532
13 VKT {"en": "Vorkuta Airport", "ru": "Воркута"} 413
14 GRV {"en": "Khankala Air Base", "ru": "Грозный"} 104
15 RTW {"en": "Saratov Central Airport", "ru": "Саратов 364
16 URS {"en": "Kursk East Airport", "ru": "Курск-Восто 208
17 YKS {"en": "Yakutsk Airport", "ru": "Якутск"} 246
18 UUS {"en": "Yuzhno-Sakhalinsk Airport", "ru": "Хом 388
19 VOG {"en": "Volgograd International Airport", "ru": " 465
20 LED {"en": "Pulkovo Airport", "ru": "Пулково"} 2,825
21 OGZ {"en": "Beslan Airport", "ru": "Беслан"} 182
22 SLY {"en": "Salekhard Airport", "ru": "Салехард"} 182
23 NJC {"en": "Nizhnevartovsk Airport", "ru": "Нижнев 284
Value
KXK

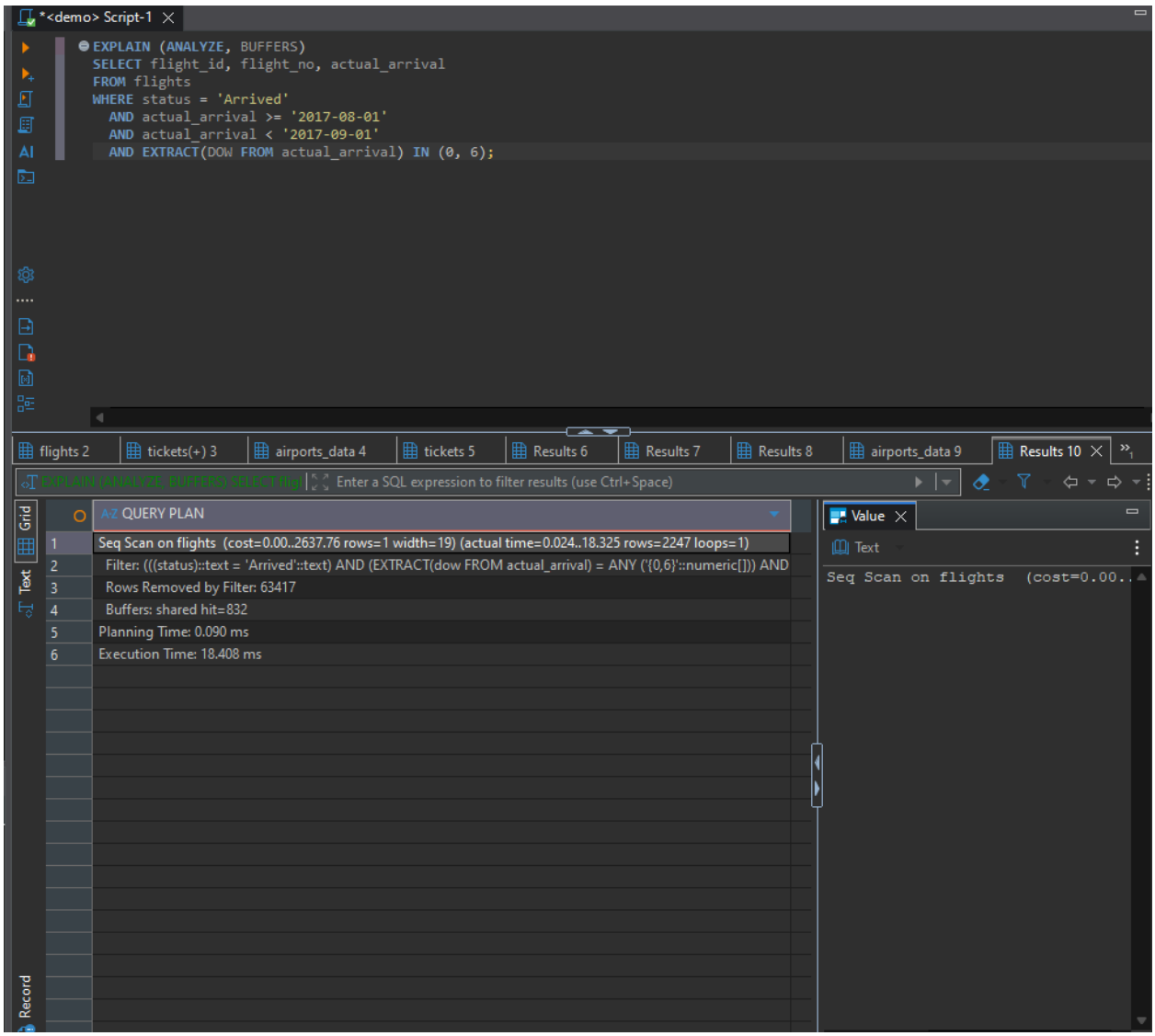


Аналитическое резюме: при выполнении запроса индексы не использовались, база данных применила полное последовательное сканирование (Seq Scan) для обеих таблиц — flights и airports_data. Обращений к физическому диску не происходило, так как все необходимые 835 блоков данных были найдены и успешно прочитаны из кэша (shared hit=835). Объединение таблиц было выполнено оптимизатором с помощью алгоритма Hash Right Join, а финальная группировка — через узел HashAggregate, при этом обе операции отработали исключительно в оперативной памяти, потребовав 18kB и 48kB соответственно

Задание 3


*<demo> Script-1
SELECT flight_id, flight_no, actual_arrival
FROM flights
WHERE status = 'Arrived'
AND EXTRACT(YEAR FROM actual_arrival) = 2017
AND EXTRACT(MONTH FROM actual_arrival) = 8
AND EXTRACT(DOW FROM actual_arrival) IN (0, 6);
airports_data 9
Results 10
Results 11
Results 12
Results 13
Results 14
Results 15
flights 16
flights 17
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
Panels
flight_id
flight_no
actual_arrival
1 41 PG0405 2017-08-13 10:34:00.000 +0300
2 42 PG0404 2017-08-13 20:02:00.000 +0300
3 43 PG0403 2017-08-13 12:23:00.000 +0300
4 44 PG0402 2017-08-13 13:25:00.000 +0300
5 107 PG0405 2017-08-06 10:35:00.000 +0300
6 108 PG0404 2017-08-06 20:00:00.000 +0300
7 113 PG0403 2017-08-06 12:22:00.000 +0300
8 114 PG0402 2017-08-06 13:23:00.000 +0300
9 202 PG0405 2017-08-12 13:37:00.000 +0300
10 203 PG0404 2017-08-12 20:02:00.000 +0300
11 204 PG0403 2017-08-12 12:23:00.000 +0300
12 205 PG0402 2017-08-12 13:23:00.000 +0300
13 421 PG0405 2017-08-05 10:34:00.000 +0300
14 426 PG0404 2017-08-05 20:01:00.000 +0300
15 432 PG0403 2017-08-05 12:28:00.000 +0300
16 433 PG0402 2017-08-05 13:20:00.000 +0300
17 489 PG0222 2017-08-13 14:32:00.000 +0300
18 492 PG0222 2017-08-05 14:30:00.000 +0300
19 553 PG0222 2017-08-12 14:36:00.000 +0300
20 573 PG0222 2017-08-06 14:35:00.000 +0300
21 608 PG0202 2017-08-06 14:26:00.000 +0300
22 701 PG0202 2017-08-12 11:37:00.000 +0300
23 713 PG0202 2017-08-13 11:38:00.000 +0300
Value
41

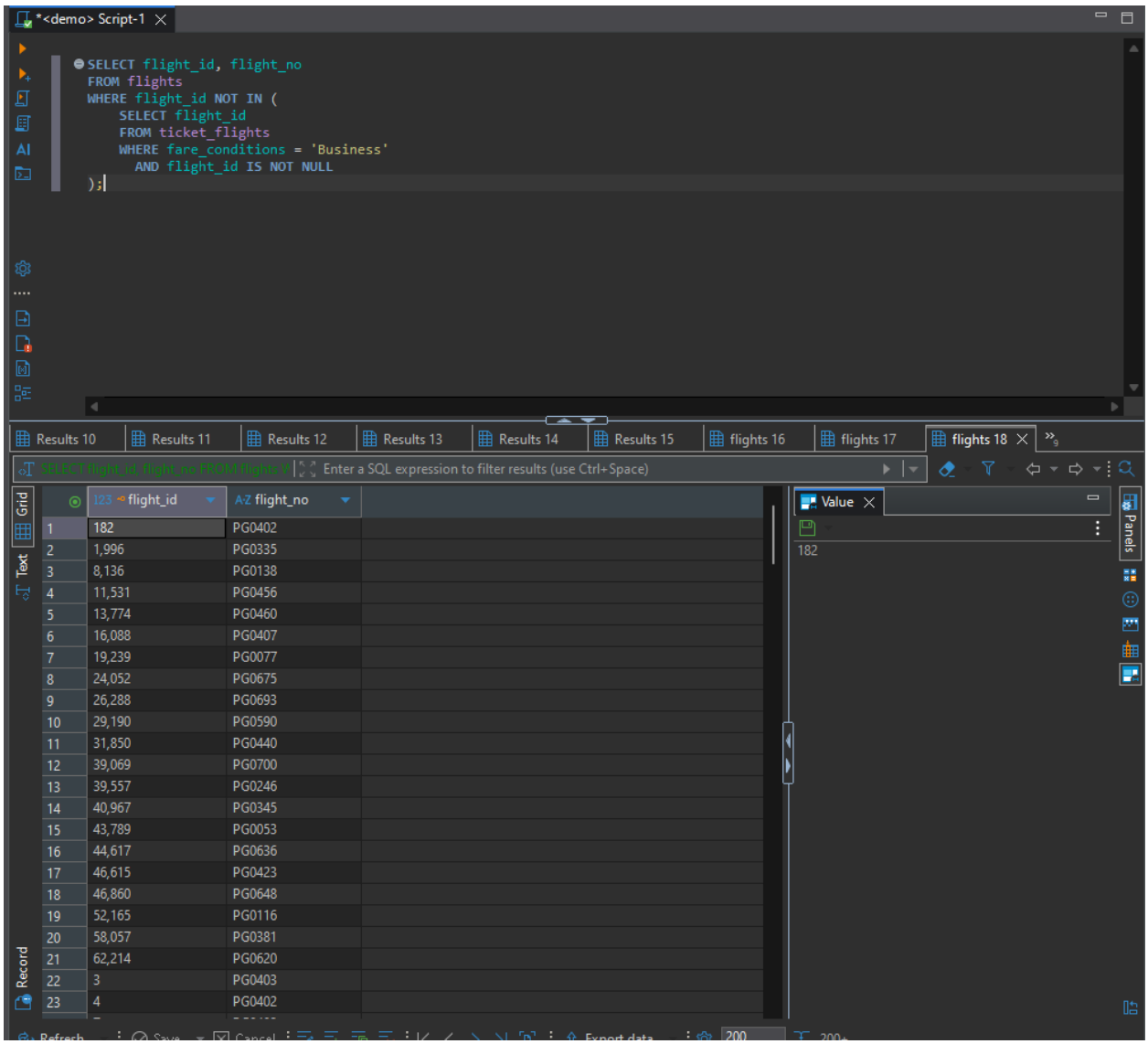

*<demo> Script-1
EXPLAIN (ANALYZE, BUFFERS)
SELECT flight_id, flight_no, actual_arrival
FROM flights
WHERE status = 'Arrived'
AND EXTRACT(YEAR FROM actual_arrival) = 2017
AND EXTRACT(MONTH FROM actual_arrival) = 8
AND EXTRACT(DOW FROM actual_arrival) IN (0, 6);
flights 2
tickets(+) 3
airports_data 4
tickets 5
Results 6
Results 7
Results 8
airports_data 9
Results 10
Enter a SQL expression to filter results (use Ctrl+Space)
QUERY PLAN
Seq Scan on flights (cost=0.00..2637.76 rows=1 width=19) (actual time=0.024..18.325 rows=2247 loops=1)
Filter: (((status)::text = 'Arrived'::text) AND (EXTRACT(dow FROM actual_arrival) = ANY ('{0,6}'::numeric[])) AND
Rows Removed by Filter: 63417
Buffers: shared hit=832
Planning Time: 0.090 ms
Execution Time: 18.408 ms
Value
Text
Seq Scan on flights (cost=0.00..
Refresh
Save
Cancel
Export data
200
MSK
en
Writable
Smart Insert
7 : 50 : 256
Sel: 0 | 0
Chat
demo
New conversation
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.
Press Ctrl+Enter to submit your query, press Enter to insert a
Поиск
13:59
11.10.2026

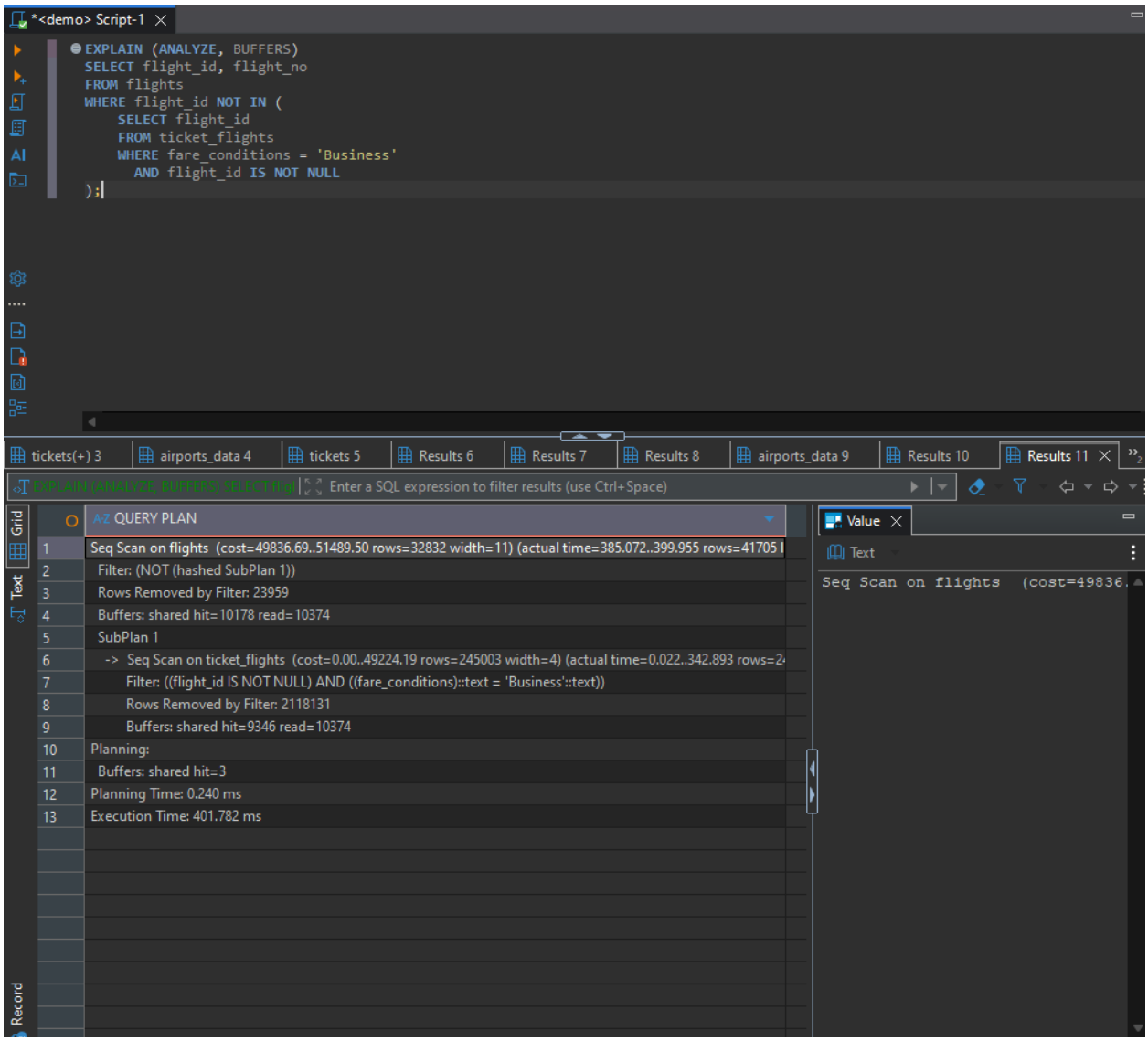


Аналитическое резюме: как неоптимизированный запрос с использованием функций EXTRACT, так и оптимизированный запрос с использованием диапазонов дат выполнили полное последовательное сканирование таблицы (Seq Scan on flights), не задействовав индексы. В обоих случаях обращения к физическому диску не было, поскольку все 832 требуемых блока данных были прочитаны напрямую из кэша (shared hit=832). В данном конкретном случае планы выполнения и показатели оказались идентичными, что говорит о том, что при отсутствии подходящего индекса на столбце actual_arrival (или из-за малого размера таблицы, полностью помещающейся в памяти) СУБД в любом случае прибегает к полному сканированию и фильтрации строк "на лету"

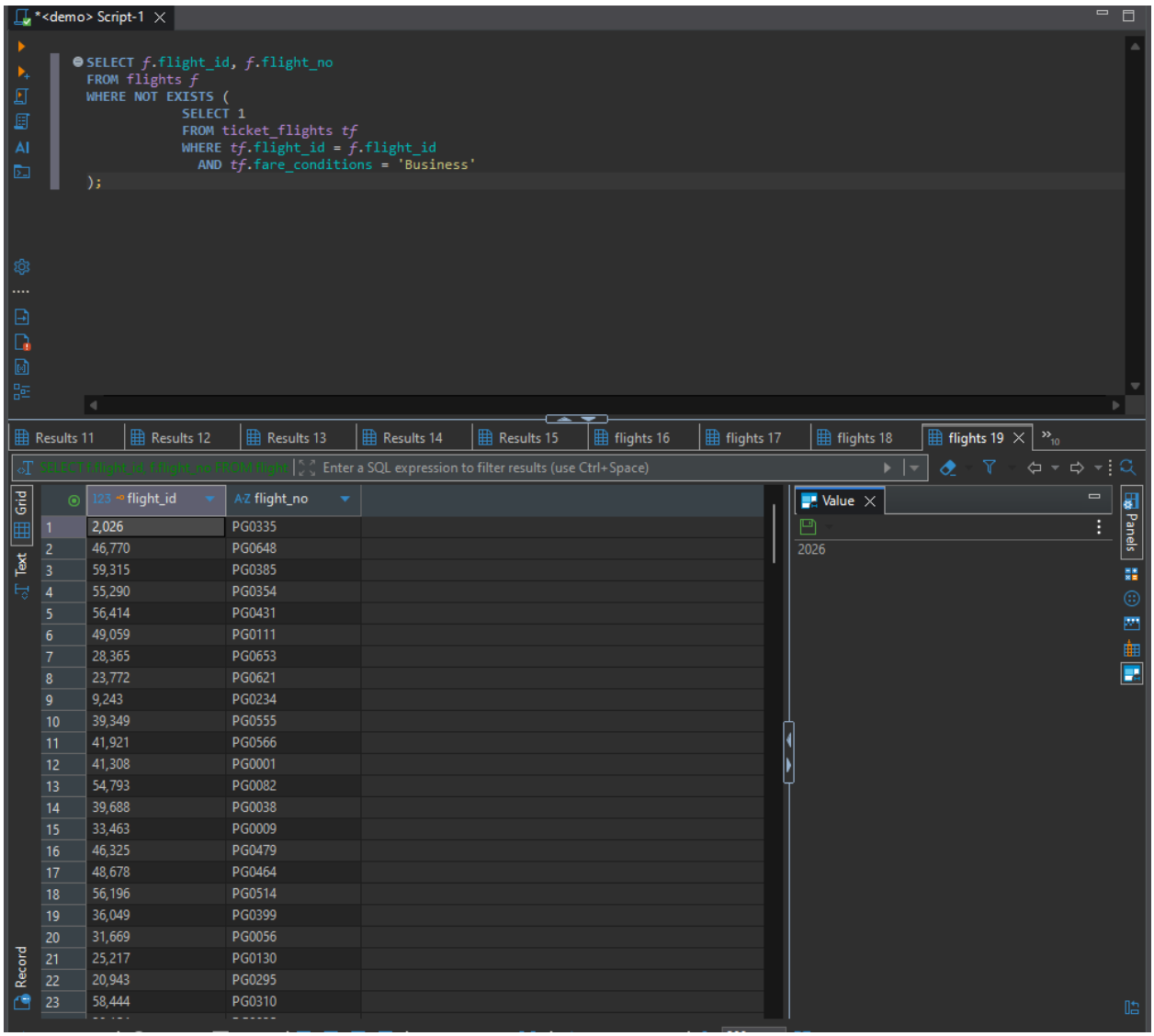
Задание 4

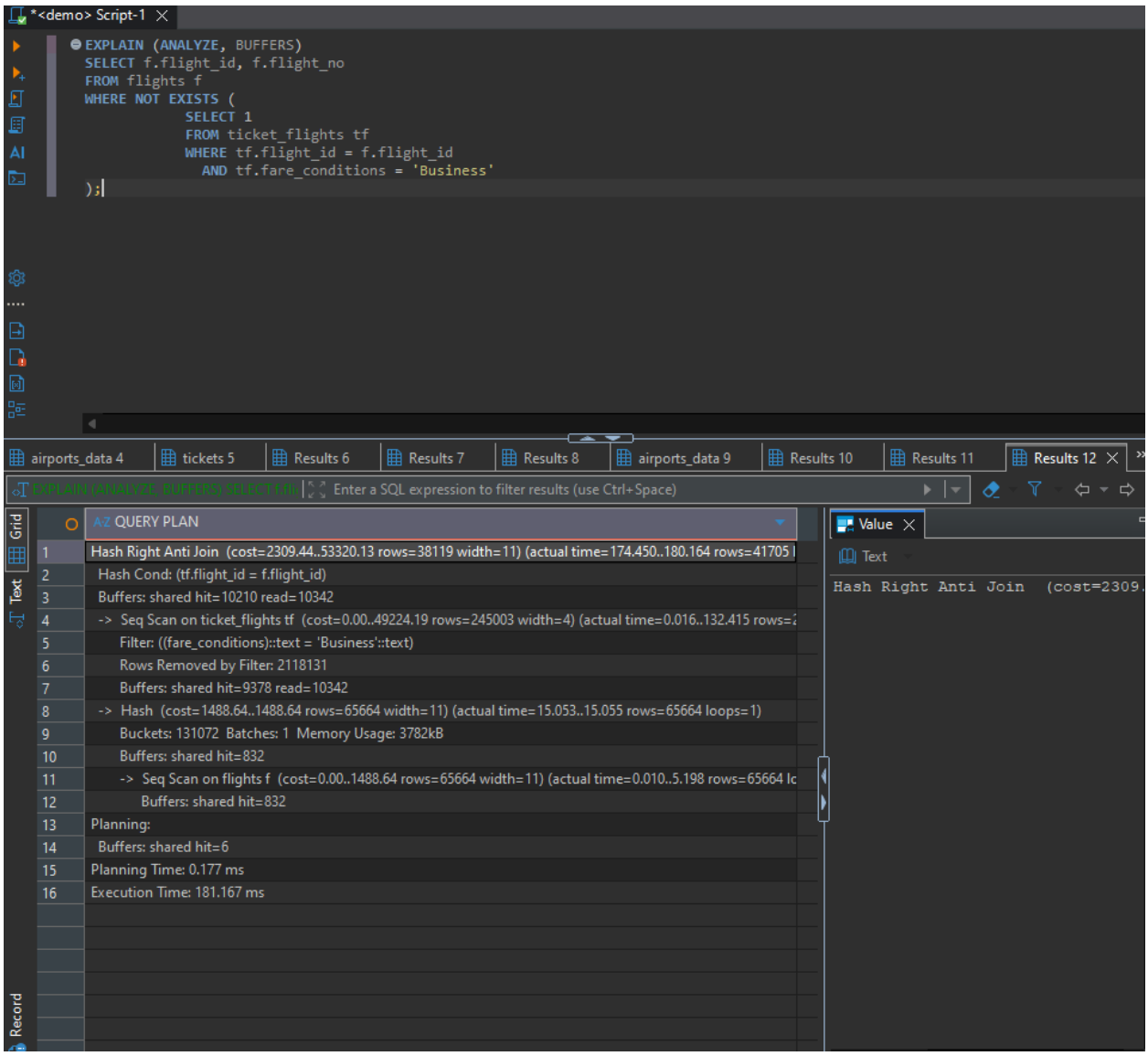
Запрос с NOT IN:


*<demo> Script-1
SELECT flight_id, flight_no
FROM flights
WHERE flight_id NOT IN (
SELECT flight_id
FROM ticket_flights
WHERE fare_conditions = 'Business'
AND flight_id IS NOT NULL
);
Results 10
Results 11
Results 12
Results 13
Results 14
Results 15
flights 16
flights 17
flights 18
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
Panels
flight_id
flight_no
1 182 PG0402
2 1,996 PG0335
3 8,136 PG0138
4 11,531 PG0456
5 13,774 PG0460
6 16,088 PG0407
7 19,239 PG0077
8 24,052 PG0675
9 26,288 PG0693
10 29,190 PG0590
11 31,850 PG0440
12 39,069 PG0700
13 39,557 PG0246
14 40,967 PG0345
15 43,789 PG0053
16 44,617 PG0636
17 46,615 PG0423
18 46,860 PG0648
19 52,165 PG0116
20 58,057 PG0381
21 62,214 PG0620
22 3 PG0403
23 4 PG0402
Value
182
Refresh
Save
Cancel
Export data
200



Запрос с NOT EXISTS:

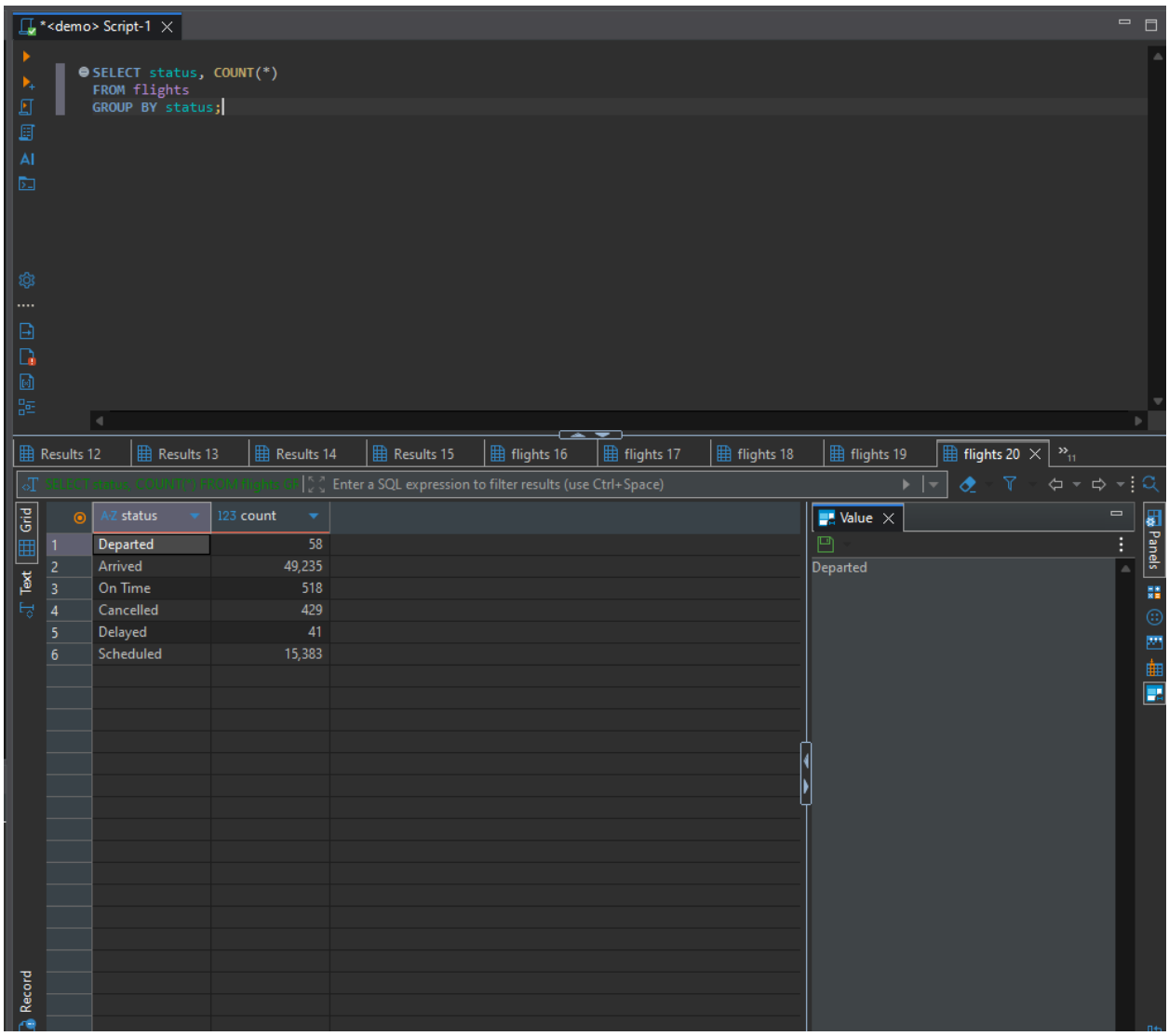

*<demo> Script-1
SELECT f.flight_id, f.flight_no
FROM flights f
WHERE NOT EXISTS (
SELECT 1
FROM ticket_flights tf
WHERE tf.flight_id = f.flight_id
AND tf.fare_conditions = 'Business'
);
Results 11
Results 12
Results 13
Results 14
Results 15
flights 16
flights 17
flights 18
flights 19
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
Panels
Value
2026
flight_id
flight_no
1 2,026 PG0335
2 46,770 PG0648
3 59,315 PG0385
4 55,290 PG0354
5 56,414 PG0431
6 49,059 PG0111
7 28,365 PG0653
8 23,772 PG0621
9 9,243 PG0234
10 39,349 PG0555
11 41,921 PG0566
12 41,308 PG0001
13 54,793 PG0082
14 39,688 PG0038
15 33,463 PG0009
16 46,325 PG0479
17 48,678 PG0464
18 56,196 PG0514
19 36,049 PG0399
20 31,669 PG0056
21 25,217 PG0130
22 20,943 PG0295
23 58,444 PG0310

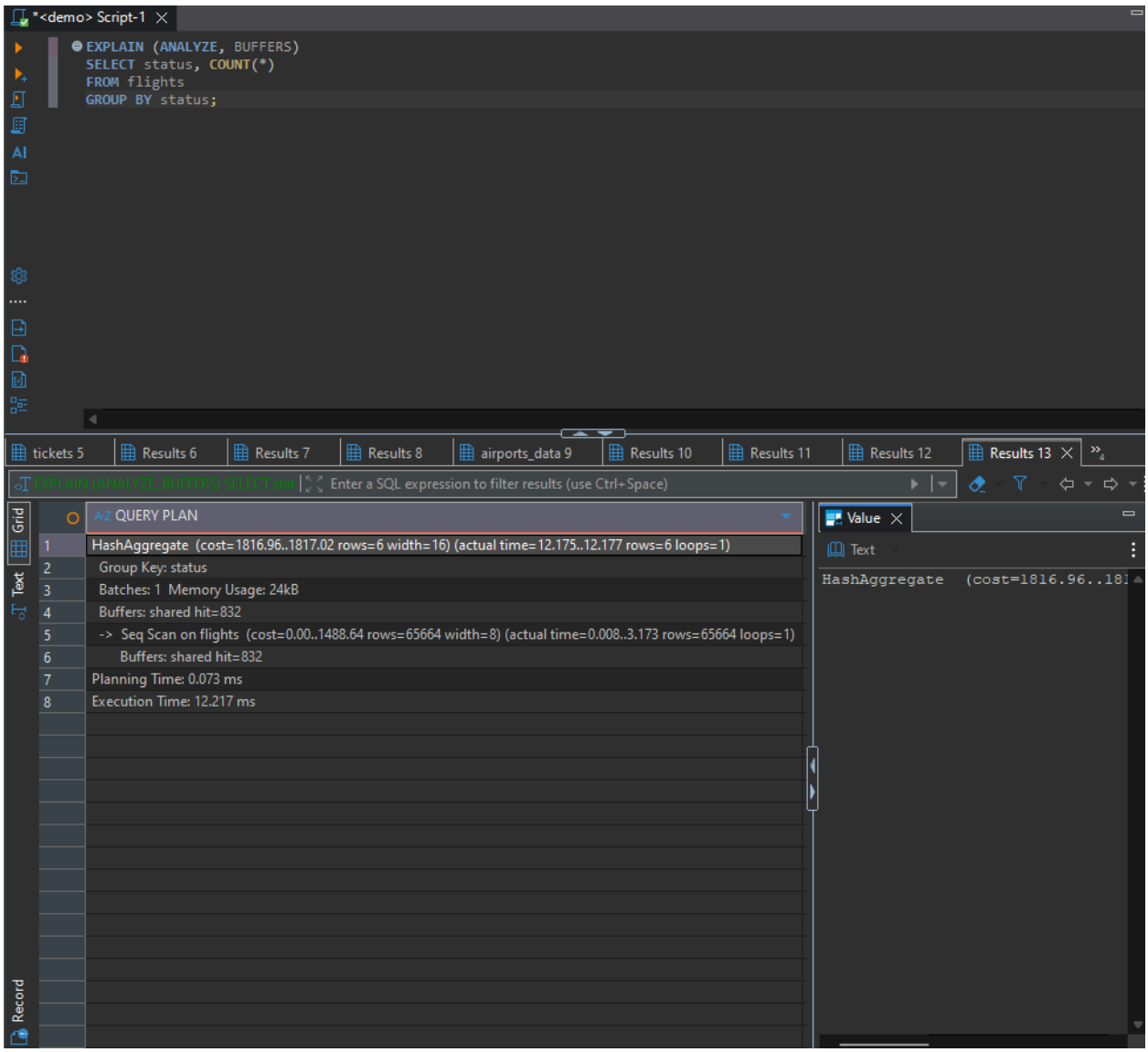


Аналитическое резюме: запрос с использованием NOT EXISTS выполнился более чем в два раза быстрее варианта с NOT IN (около 181 мс против 402 мс) за счет применения эффективного алгоритма Hash Right Anti Join. В обоих случаях индексы не использовались, и СУБД была вынуждена прибегнуть к полному последовательному сканированию (Seq Scan) таблиц flights и ticket_flights. Оба запроса активно работали не только с кэшем (более 9-10 тысяч shared hit), но и с физическим диском, прочитав свыше 10 тысяч блоков данных напрямую с накопителя (read). При использовании NOT EXISTS планировщик смог эффективно разместить хэш-таблицу в оперативной памяти (потребовалось около 3782kB), что позволило быстрее отфильтровать ненужные строки

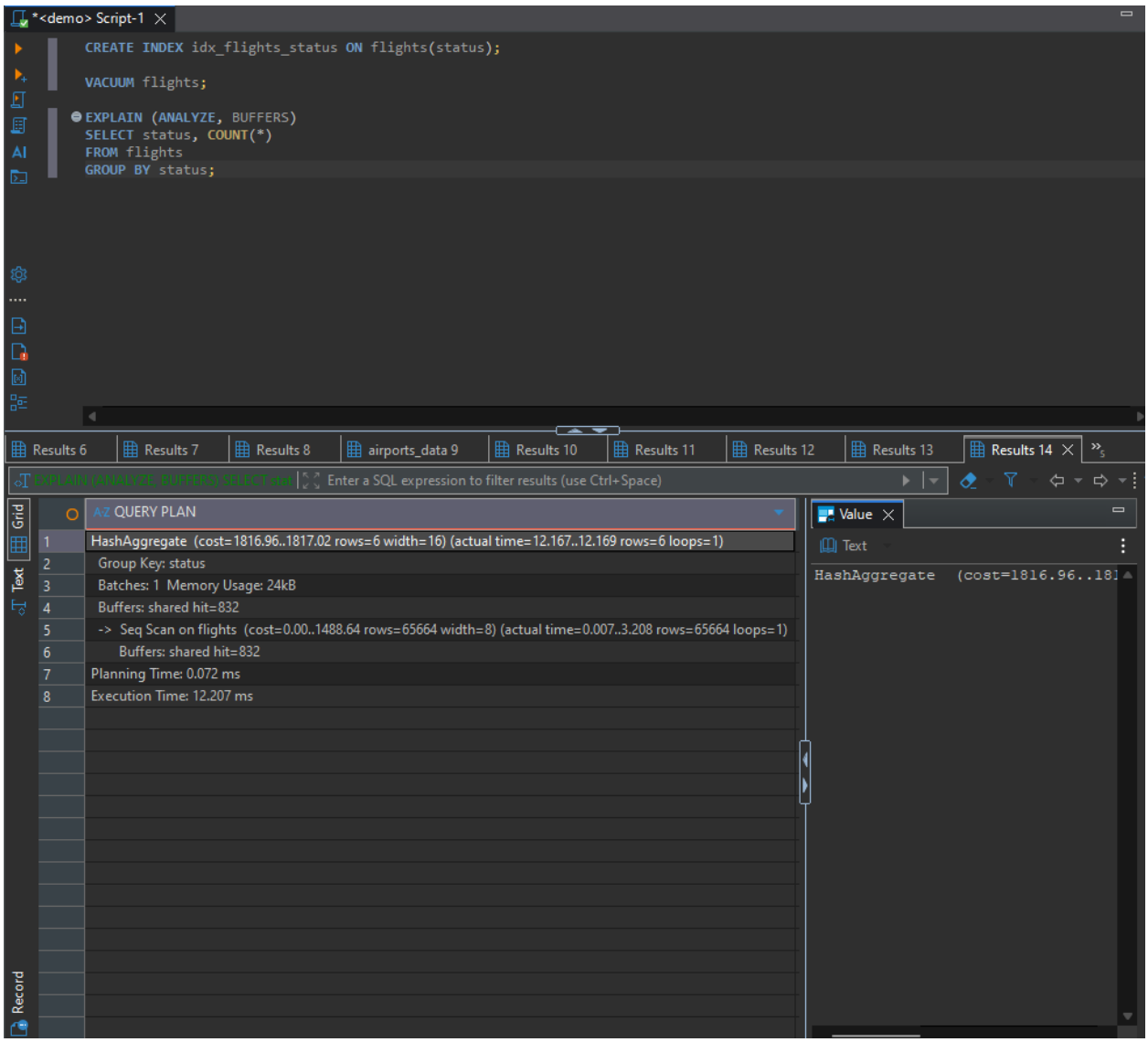
Задание 5

Без индексов


*<demo> Script-1
SELECT status, COUNT(*)
FROM flights
GROUP BY status;
Results 12
Results 13
Results 14
Results 15
flights 16
flights 17
flights 18
flights 19
flights 20
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
Panels
status
count
1 Departed 58
2 Arrived 49,235
3 On Time 518
4 Cancelled 429
5 Delayed 41
6 Scheduled 15,383
Value
Departed



С покрывающим индексом



Аналитическое резюме: несмотря на создание покрывающего индекса и выполнение VACUUM, планировщик базы данных в обоих случаях предпочел не использовать индекс, применив полное последовательное сканирование таблицы (Seq Scan on flights). Обращения к физическому диску не происходило ни до, ни после добавления индекса, так как все необходимые 832 блока данных уже находились и читались напрямую из кэша (shared hit=832). Оптимизатор PostgreSQL проигнорировал новый индекс, поскольку таблица имеет небольшой размер и полностью помещается в оперативной памяти, из-за чего прямое чтение из кэша с последующей группировкой HashAggregate оказалось для СУБД "дешевле" и эффективнее переходов по структуре индекса

Index Only Scan — это метод выполнения запроса, при котором база данных извлекает все нужные данные исключительно из индекса, вообще не обращаясь к основной таблице (heap)

Это значительно ускоряет работу благодаря Visibility Map (карте видимости). Поскольку в PostgreSQL могут храниться устаревшие версии строк, база данных должна проверять их актуальность. Visibility Map — это компактная структура, которая отмечает, содержит ли

блок таблицы только актуальные записи. Если да, СУБД доверяет данным из индекса и пропускает чтение тяжелой основной таблицы, экономя время и ресурсы.

Команда VACUUM регулярно обновляет эту карту, позволяя базе безопасно использовать Index Only Scan